

Higher-Order Finite Difference Methods for Quantum Eigenvalue
Problems:
Convergence and Efficiency of Second- and Fourth-Order
Discretizations

Joshua Hiller

March 16, 2026

Contents

Introduction	3
1 The Finite Difference Method	4
1.1 Introduction to Finite Difference Methods	4
1.2 Grid Construction and Boundary Conditions	5
1.2.1 Uniform Interior Grid	5
1.3 Second-Order Finite Difference Method	5
1.3.1 Taylor Series Expansion and Coefficient Derivation	6
1.3.2 Matrix Assembly for Second-Order Method	7
1.3.3 Boundary Conditions for Second-Order Method	7
1.4 Fourth-Order Finite Difference Method	8
1.4.1 Taylor Series Expansion and Coefficient Derivation	8
1.4.2 Fourth-Order Matrix Assembly	9
1.4.3 Boundary Conditions for Fourth-Order Method	10
1.5 Performance Analysis and Computational Efficiency	11
1.5.1 Sparse Matrix Storage and Computational Scaling Analysis	11
1.5.2 Accuracy and Convergence Behavior	12
1.5.3 Systematic Error Analysis	13
1.5.4 Unified Implementation Architecture	13
1.5.5 The Accuracy-Efficiency Trade-off	13
2 Applications in Quantum Mechanics	15
2.1 Introduction to Quantum Eigenvalue Problems	15
2.1.1 The Time-Independent Schrödinger Equation	15
2.1.2 General Nondimensionalization Procedure	15
2.2 The Infinite Square Well	16
2.2.1 Problem Statement and Nondimensionalization	16
2.2.2 Analytical Solution	17
2.2.3 Spectral Features and Discretization Implications	19
2.3 The Quantum Simple Harmonic Oscillator	19
2.3.1 Problem Statement and Nondimensionalization	20
2.3.2 Analytical Solution	21
2.3.3 Spectral Features and Discretization Implications	22
3 Results and Analysis	24
3.1 Runtime and Efficiency	24
3.2 Convergence Studies	24
3.3 Mode Index vs Error	27
3.3.1 QSHO Spectrum Validation	28
3.3.2 QSHO Domain Truncation Sensitivity	28

4	Conclusions and Future Work	30
4.1	Summary of Results	30
4.2	Future Work	30
A	Implementation Details	31
A.1	Implementation Details for Fourth-Order Boundary Corrections	31
A.1.1	Implementation Considerations	31
A.1.2	Implementation Algorithm	31
A.1.3	Matrix Verification Methodology	32
A.2	Unified Implementation Architecture	32
A.2.1	Coefficient Management System	32
A.2.2	Unified Diagonal Construction	32
A.2.3	Unified Matrix Assembly	32
A.2.4	Unified Boundary Corrections	33
A.2.5	Unified Hamiltonian Construction	33
A.2.6	Code Architecture Benefits	33
A.2.7	Implementation Algorithm	33
A.3	ISW Implementation	34
A.4	Python Module Summaries	34
A.4.1	<code>finite_difference_quantum.py</code>	34
A.4.2	<code>eigenvalue_analysis.py</code>	35
A.4.3	<code>n_break_test.py</code>	35
A.5	Figure Plotting Specifications	35

Introduction

This thesis is about using a computational method called the finite difference method to solve quantum mechanics problems. This method turns calculus equations into matrix equations that only a computer can solve. The main question is whether using a more advanced version of the method is worth the extra work, based on how much better it performs.

The core objectives are:

- build two versions of the method (second-order and fourth-order) and apply the boundary conditions correctly,
- check that the numerical results approach known exact solutions as the grid is refined, and
- compare accuracy and computation time for two test problems: the Infinite Square Well (ISW) and the Quantum Simple Harmonic Oscillator (QSHO).

The presentation is organized as follows. Chapter 1 develops the finite difference method, including grid construction, matrix assembly, and boundary closures. Chapter 2 develops the benchmark quantum models at the theoretical level and derives analytical solutions for direct comparison. Chapter 3 presents numerical results and analysis, including convergence, accuracy, and runtime studies.

Chapter 1

The Finite Difference Method

1.1 Introduction to Finite Difference Methods

The finite difference method transforms differential equation problems into finite-dimensional linear algebra problems [1]. For eigenvalue equations, this reduction yields matrix eigenvalue problems. This approach is fundamental to computational science and provides a systematic way to solve complex differential equations numerically [2]. Consider the general linear eigenvalue problem described by the second-order differential equation

$$A(x)\psi'' + B(x)\psi' + C(x)\psi = \lambda\psi, \quad (1.1)$$

on the interval $x \in [x_{\text{left}}, x_{\text{right}}]$, where $A(x)$, $B(x)$, and $C(x)$ are known coefficient functions, and we seek eigenvalues λ and eigenfunctions $\psi(x)$. We capture this compactly by defining the linear differential operator

$$(\mathcal{M}\psi)(x) = A(x)\psi''(x) + B(x)\psi'(x) + C(x)\psi(x), \quad (1.2)$$

so that the eigenvalue problem can be written succinctly as $\mathcal{M}\psi = \lambda\psi$.

Problems of this form, together with boundary conditions, are ubiquitous in quantum mechanics: the allowed energies arise as eigenvalues, and the corresponding wavefunctions are the associated eigenfunctions.

The complete eigenvalue problem includes boundary conditions at the endpoints:

$$\psi(x_{\text{left}}) = 0, \quad \psi(x_{\text{right}}) = 0 \quad (1.3)$$

These parallel Dirichlet boundary conditions ensure that the wavefunction vanishes at the boundaries, which is appropriate for many physical systems, including particles confined to a finite region [3].

The finite difference approach begins by replacing the continuous interval $[x_{\text{left}}, x_{\text{right}}]$ with a finite spatial grid $\{x_1, x_2, \dots, x_N\}$. The wavefunction is then represented by sampled values on this grid, collected into a vector ψ , and the differential operator $\mathcal{M}$ is replaced by a matrix M acting on that vector [4]. This transforms the original differential eigenvalue problem into a matrix eigenvalue problem:

$$M\psi = \lambda\psi \quad (1.4)$$

where the matrix M approximates the differential operator $\mathcal{M}$ on the discrete grid.

The choice of computational grid determines the fundamental accuracy characteristics. This includes both the spacing between grid points Δ and, in some cases, the choice of domain endpoints $[x_{\text{left}}, x_{\text{right}}]$. Finer grids (smaller Δ) generally provide better approximations to the continuous differential operator.

There is more than one way to construct finite-difference formulas on the same grid. In this thesis we compare second-order and fourth-order schemes. At a qualitative level, both improve as the grid is refined, but the fourth-order method typically gains accuracy faster with refinement [1, 5].

This chapter is organized as follows: Section 1.2 develops the fundamental grid construction and boundary condition implementation. Section 1.3 derives the second-order finite difference method, including Taylor series expansions and matrix assembly. Section 1.4 extends this approach to fourth-order accuracy, highlighting the increased complexity of boundary treatments. Finally, Section 1.5 discusses performance analysis and computational efficiency considerations.

1.2 Grid Construction and Boundary Conditions

1.2.1 Uniform Interior Grid

As shown in Figure 1.1, the finite difference method discretizes the continuous domain $[x_{\text{left}}, x_{\text{right}}]$ using a uniform interior grid. On this grid, the nodes and spacing are defined as

$$\begin{aligned} x_i &= x_{\text{left}} + i \Delta, \quad i = 1, \dots, N, \\ \Delta &= \frac{x_{\text{right}} - x_{\text{left}}}{N + 1}. \end{aligned} \quad (1.5)$$

This grid includes only interior points. The boundary values are known, so they are not part of the unknown solution. The finite difference method approximates the wavefunction at each interior grid point, while boundary values are imposed separately.

The boundary conditions in terms of the grid are:

$$\psi_0 = \psi_{N+1} = 0 \quad (1.6)$$

where ψ_0 and ψ_{N+1} represent the wavefunction values at the boundary points outside the interior grid.

Associated with this grid, we define the unknown "wavefunction vector" that collects only interior samples:

$$\boldsymbol{\psi} = (\psi_1, \dots, \psi_N)^\top, \quad (1.7)$$

where $\psi_i \equiv \psi(x_i)$ represents the wavefunction value at grid point x_i .

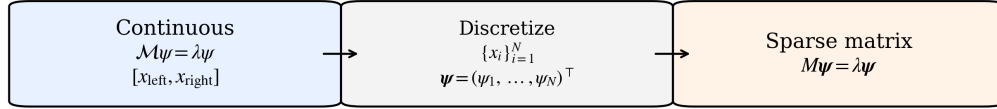


Figure 1.1: Schematic of a uniform interior grid on $[x_{\text{left}}, x_{\text{right}}]$ with spacing Δ and interior nodes $x_1, \dots, x_N$ as shown in [6].

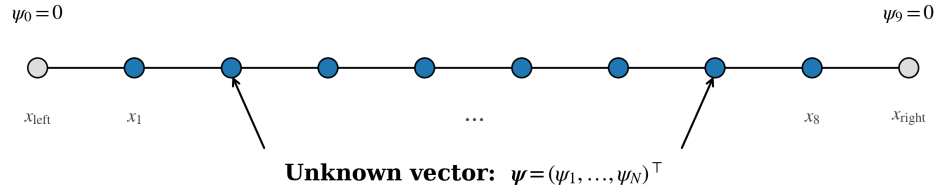


Figure 1.2: Schematic emphasizing *interior-only* unknowns

1.3 Second-Order Finite Difference Method

This section develops the second-order finite difference method for solving eigenvalue problems. The approach consists of three main components: First, we derive finite difference approximations for derivatives using

Taylor series expansions, achieving second-order accuracy in the grid spacing Δ . Second, we assemble these approximations into a matrix representation of the differential operator. Finally, we implement the boundary conditions to complete the discrete eigenvalue problem. The result is a banded matrix eigenvalue problem that approximates the original continuous differential equation.

1.3.1 Taylor Series Expansion and Coefficient Derivation

General Taylor Series Formula. The Taylor series expansion of a function $f(x_0 + \epsilon)$ about a point x_0 is given by:

$$f(x_0 + \epsilon) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!} \epsilon^n = f(x_0) + f'(x_0)\epsilon + \frac{f''(x_0)}{2!} \epsilon^2 + \frac{f'''(x_0)}{3!} \epsilon^3 + \dots \quad (1.8)$$

This general formula will be applied with different choices of ϵ corresponding to grid point displacements.

Applying Equation (1.8) with $\epsilon = \pm\Delta$ gives the well-known expansions (for small $\Delta > 0$) [2]

$$\psi(x + \Delta) = \psi(x) + \psi'(x)\Delta + \frac{1}{2}\psi''(x)\Delta^2 + \frac{1}{6}\psi^{(3)}(x)\Delta^3 + \frac{1}{24}\psi^{(4)}(x)\Delta^4 + \mathcal{O}(\Delta^5), \quad (1.9)$$

$$\psi(x - \Delta) = \psi(x) - \psi'(x)\Delta + \frac{1}{2}\psi''(x)\Delta^2 - \frac{1}{6}\psi^{(3)}(x)\Delta^3 + \frac{1}{24}\psi^{(4)}(x)\Delta^4 + \mathcal{O}(\Delta^5). \quad (1.10)$$

The notation $\mathcal{O}(\Delta^n)$ indicates that the truncation error scales as the n th power of the grid spacing, meaning the error decreases as Δ becomes smaller. In this context, “second-order” means the leading truncation error term scales as Δ^2 .

Centered first derivative. Subtracting Equation (1.10) from Equation (1.9) gives

$$\psi'(x) = \frac{\psi(x + \Delta) - \psi(x - \Delta)}{2\Delta} + \mathcal{O}(\Delta^2), \quad (1.11)$$

where the truncation error is second-order in Δ .

Centered second derivative. Adding Equations (1.9) and (1.10) yields

$$\psi''(x) = \frac{\psi(x + \Delta) - 2\psi(x) + \psi(x - \Delta)}{\Delta^2} + \mathcal{O}(\Delta^2), \quad (1.12)$$

again giving second-order accuracy in Δ .

Grid formulas. Evaluated at interior nodes x_i , these become

$$\psi'_i = \frac{\psi_{i+1} - \psi_{i-1}}{2\Delta}, \quad \psi''_i = \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta^2}. \quad (1.13)$$

Centered schemes achieve second-order accuracy because symmetry cancels odd powers of Δ . By contrast, one-sided differences are first order; for example,

$$\frac{\psi(x + \Delta) - \psi(x)}{\Delta} = \psi'(x) + \frac{1}{2}\psi''(x)\Delta + \mathcal{O}(\Delta^2). \quad (1.14)$$

So forward differences approximate $\psi'(x)$ with $\mathcal{O}(\Delta)$ error.

These finite difference formulas have intuitive interpretations. The first derivative formula Equation (1.11) represents the slope of the secant line connecting the points at $x \pm \Delta$, which approximates the tangent line slope (the derivative) at x . The second derivative formula Equation (1.12) can be understood as the discrete analog of “the derivative of a derivative”. It measures how much the slope changes from one side of the point to the other. The centered approach provides higher accuracy because it treats both directions symmetrically, automatically canceling out the leading error terms.

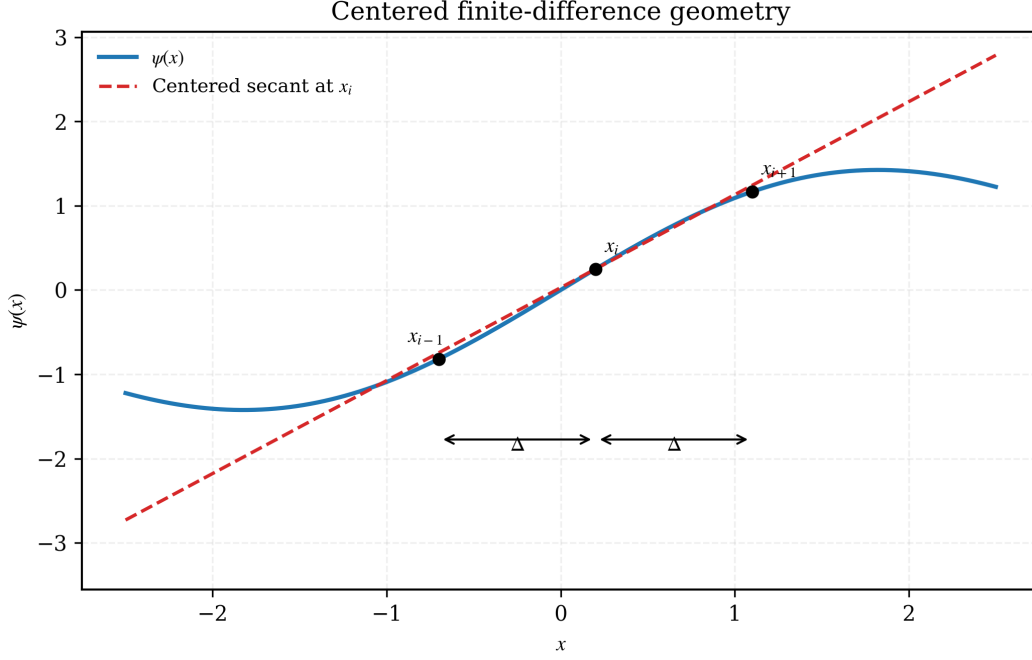


Figure 1.3: Visual interpretation of centered finite differences.

1.3.2 Matrix Assembly for Second-Order Method

Consider the generic linear operator

$$\mathcal{M}\psi = A(x)\psi'' + B(x)\psi' + C(x)\psi. \quad (1.15)$$

Projecting A, B, C onto the grid ($A_i = A(x_i)$, etc.) and inserting Equation (1.13) yields the corresponding three-point finite-difference equation [7]

$$\left(\frac{A_i}{\Delta^2} + \frac{B_i}{2\Delta}\right)\psi_{i+1} + \left(C_i - \frac{2A_i}{\Delta^2}\right)\psi_i + \left(\frac{A_i}{\Delta^2} - \frac{B_i}{2\Delta}\right)\psi_{i-1} = \lambda\psi_i. \quad (1.16)$$

Thus the matrix M in Equation (1.4) has entries

$$M_{i,i\pm 1} = \left(\frac{A_i}{\Delta^2} \pm \frac{B_i}{2\Delta}\right), \quad M_{ii} = C_i - \frac{2A_i}{\Delta^2}, \quad (1.17)$$

with all other $M_{ij} = 0$, so only the main diagonal and first off-diagonals are nonzero (a tridiagonal band structure; see Figure 1.4), symmetric when $B = 0$ [4].

Each interior equation couples only to nearest neighbors, so the matrix has bandwidth one and only $\mathcal{O}(N)$ nonzeros.

1.3.3 Boundary Conditions for Second-Order Method

Using Equation (1.6), endpoint values are fixed to zero ($\psi_0 = \psi_{N+1} = 0$), so only interior values in Equation (1.7) remain unknown.

For the first row ($i = 1$), the complete stencil equation initially includes all three terms:

$$\left(\frac{A_1}{\Delta^2} - \frac{B_1}{2\Delta}\right)\underbrace{\psi_0}_{=0} + \left(C_1 - \frac{2A_1}{\Delta^2}\right)\psi_1 + \left(\frac{A_1}{\Delta^2} + \frac{B_1}{2\Delta}\right)\psi_2 = \lambda\psi_1, \quad (1.18)$$

where the first term vanishes because $\psi_0 = 0$ due to the homogeneous Dirichlet boundary condition. This simplifies to:

$$\left(C_1 - \frac{2A_1}{\Delta^2}\right)\psi_1 + \left(\frac{A_1}{\Delta^2} + \frac{B_1}{2\Delta}\right)\psi_2 = \lambda\psi_1. \quad (1.19)$$

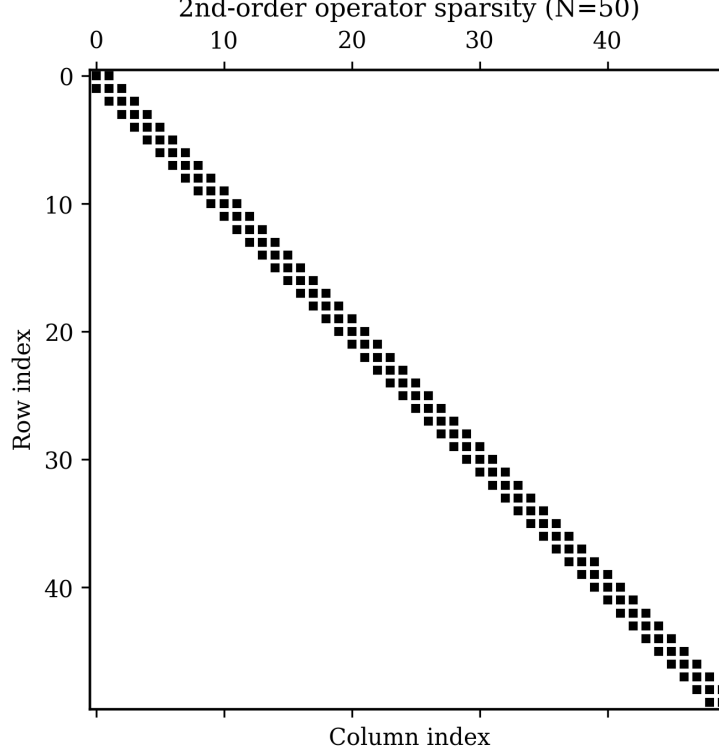


Figure 1.4: Sparsity pattern of the second-order tridiagonal matrix for a representative N ($N = 50$), illustrating the three nonzero diagonals.

Similarly, for the last row ($i = N$), the boundary condition $\psi_{N+1} = 0$ eliminates the corresponding term. This well-known approach preserves the tridiagonal structure while properly implementing the boundary conditions. (For higher-order stencils, special boundary formulas are required; these are discussed in a later section.) [8]

1.4 Fourth-Order Finite Difference Method

Having established the second-order finite difference method, we now extend the approach to achieve fourth-order accuracy. The fundamental strategy remains the same: derive finite difference approximations using Taylor series, assemble these into a matrix representation, and implement boundary conditions. What differs in the fourth-order case is the increased stencil size (five points instead of three), the resulting matrix structure (pentadiagonal instead of tridiagonal), and the more complex boundary treatment required to maintain high-order accuracy at the domain boundaries.

1.4.1 Taylor Series Expansion and Coefficient Derivation

To achieve fourth-order accuracy, we extend the Taylor series to include more terms, following well-established methods [1, 9]:

$$\psi(x + \Delta) = \psi(x) + \psi'(x)\Delta + \frac{1}{2}\psi''(x)\Delta^2 + \frac{1}{6}\psi^{(3)}(x)\Delta^3 + \frac{1}{24}\psi^{(4)}(x)\Delta^4 + \frac{1}{120}\psi^{(5)}(x)\Delta^5 + \mathcal{O}(\Delta^6), \quad (1.20)$$

$$\psi(x - \Delta) = \psi(x) - \psi'(x)\Delta + \frac{1}{2}\psi''(x)\Delta^2 - \frac{1}{6}\psi^{(3)}(x)\Delta^3 + \frac{1}{24}\psi^{(4)}(x)\Delta^4 - \frac{1}{120}\psi^{(5)}(x)\Delta^5 + \mathcal{O}(\Delta^6), \quad (1.21)$$

$$\psi(x + 2\Delta) = \psi(x) + 2\psi'(x)\Delta + 2\psi''(x)\Delta^2 + \frac{4}{3}\psi^{(3)}(x)\Delta^3 + \frac{2}{3}\psi^{(4)}(x)\Delta^4 + \frac{4}{15}\psi^{(5)}(x)\Delta^5 + \mathcal{O}(\Delta^6), \quad (1.22)$$

$$\psi(x - 2\Delta) = \psi(x) - 2\psi'(x)\Delta + 2\psi''(x)\Delta^2 - \frac{4}{3}\psi^{(3)}(x)\Delta^3 + \frac{2}{3}\psi^{(4)}(x)\Delta^4 - \frac{4}{15}\psi^{(5)}(x)\Delta^5 + \mathcal{O}(\Delta^6). \quad (1.23)$$

Centered fourth-order second derivative. By combining these expansions in the appropriate linear combination, we eliminate the leading error terms to achieve fourth-order accuracy. For the second derivative:

$$\psi''(x) = \frac{-\psi(x+2\Delta) + 16\psi(x+\Delta) - 30\psi(x) + 16\psi(x-\Delta) - \psi(x-2\Delta)}{12\Delta^2} + \mathcal{O}(\Delta^4). \quad (1.24)$$

The coefficient pattern $(-1/12, 4/3, -5/2, 4/3, -1/12)$ is selected so lower-order truncation terms cancel, leaving a leading $\mathcal{O}(\Delta^4)$ error term.

Centered fourth-order first derivative. Similarly, for the first derivative:

$$\psi'(x) = \frac{-\psi(x+2\Delta) + 8\psi(x+\Delta) - 8\psi(x-\Delta) + \psi(x-2\Delta)}{12\Delta} + \mathcal{O}(\Delta^4). \quad (1.25)$$

Again, the centered approach with coefficients $(1/12, -2/3, 0, 2/3, -1/12)$ achieves fourth-order accuracy by systematic cancellation of lower-order error terms.

Grid formulas. Evaluated at interior nodes x_i , these become:

$$\psi'_i = \frac{-\psi_{i+2} + 8\psi_{i+1} - 8\psi_{i-1} + \psi_{i-2}}{12\Delta}, \quad (1.26)$$

$$\psi''_i = \frac{-\psi_{i+2} + 16\psi_{i+1} - 30\psi_i + 16\psi_{i-1} - \psi_{i-2}}{12\Delta^2}. \quad (1.27)$$

1.4.2 Fourth-Order Matrix Assembly

Applying the fourth-order finite difference formulas Equation (1.26) to the generic linear operator Equation (1.15), and sampling A, B, C on the grid, yields a five-point stencil:

$$\left(\frac{-A_i}{12\Delta^2} + \frac{B_i}{12\Delta}\right)\psi_{i-2} + \left(\frac{16A_i}{12\Delta^2} - \frac{8B_i}{12\Delta}\right)\psi_{i-1} + \left(C_i - \frac{30A_i}{12\Delta^2}\right)\psi_i + \left(\frac{16A_i}{12\Delta^2} + \frac{8B_i}{12\Delta}\right)\psi_{i+1} + \left(\frac{-A_i}{12\Delta^2} - \frac{B_i}{12\Delta}\right)\psi_{i+2} = \lambda\psi_i. \quad (1.28)$$

Thus the matrix M in $M\psi = \lambda\psi$ has entries

$$M_{i,i\pm 2} = \left(\frac{-A_i}{12\Delta^2} \mp \frac{B_i}{12\Delta}\right), \quad M_{i,i\pm 1} = \left(\frac{16A_i}{12\Delta^2} \pm \frac{8B_i}{12\Delta}\right), \quad M_{ii} = C_i - \frac{30A_i}{12\Delta^2}, \quad (1.29)$$

with all other $M_{ij} = 0$, a well-known pentadiagonal band structure (symmetric when $B = 0$) [8].

The pentadiagonal structure is illustrated in Figure 1.5 for $N = 7$ interior points.

Fourth-Order Pentadiagonal Matrix (N=7):

$$M_{\text{pentadiagonal}} = \begin{bmatrix} d_1 & u_1 & v_1 & 0 & 0 & 0 & 0 \\ w_1 & d_2 & u_2 & v_2 & 0 & 0 & 0 \\ x_1 & w_2 & d_3 & u_3 & v_3 & 0 & 0 \\ 0 & x_2 & w_3 & d_4 & u_4 & v_4 & 0 \\ 0 & 0 & x_3 & w_4 & d_5 & u_5 & v_5 \\ 0 & 0 & 0 & x_4 & w_5 & d_6 & u_6 \\ 0 & 0 & 0 & 0 & x_5 & w_6 & d_7 \end{bmatrix}$$

where d_i (diagonal), u_i (upper 1), v_i (upper 2), w_i (lower 1), x_i (lower 2) give a total of $5N - 4 = 31$ nonzero entries. The pentadiagonal structure has two diagonals on each side of the main diagonal.

Figure 1.5: Fourth-order finite difference pentadiagonal matrix structure for $N = 7$ interior points, showing 31 nonzero entries before boundary corrections.

The comparison in Figure 1.6 vs. Figure 1.4 makes the accuracy-cost trade-off concrete: fourth-order increases stencil width (and therefore nonzeros per row), but preserves linear-in- N sparsity. In practice, this wider stencil is typically offset by substantially improved accuracy per grid point.

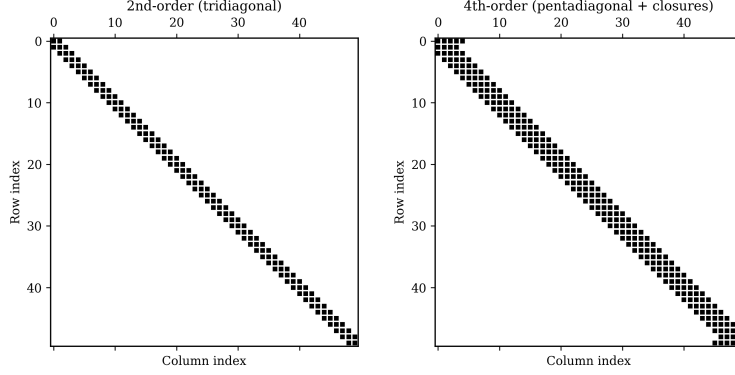


Figure 1.6: Sparsity pattern comparison highlighting the wider fourth-order band structure relative to second order.

1.4.3 Boundary Conditions for Fourth-Order Method

The fourth-order method uses the centered five-point interior formula Equation (1.26). At the first interior node this gives

$$\psi''(x_1) = \frac{-\psi_{-1} + 16\psi_0 - 30\psi_1 + 16\psi_2 - \psi_3}{12\Delta^2} + \mathcal{O}(\Delta^4). \quad (1.30)$$

Even after applying $\psi_0 = 0$, the term ψ_{-1} lies outside the computational unknowns, so the centered interior row cannot be used directly at x_1 (and symmetrically at x_N). For fourth-order accuracy, the boundary-adjacent rows therefore require one-sided closures that use additional in-domain points. In this implementation, the first/last interior rows use six-point closures, while the second/second-to-last rows use shifted five-point formulas with boundary values eliminated by $\psi_0 = \psi_{N+1} = 0$.

Indexing and notation. Let the unknown vector be $\boldsymbol{\psi} = (\psi_1, \dots, \psi_N)^\top$ on the interior grid $x_i = x_{\text{left}} + i\Delta$, $i = 1, \dots, N$, with homogeneous Dirichlet data $\psi_0 = \psi_{N+1} = 0$.

Closure at the first interior node x_1 (Taylor matching). We seek a one-sided *six-point* approximation for $\psi''(x_1)$ using the boundary node x_0 and the next five grid nodes:

$$\psi''(x_1) \approx \frac{1}{\Delta^2} \sum_{m=-1}^4 b_m \psi(x_1 + m\Delta) = \frac{1}{\Delta^2} (b_{-1}\psi_0 + b_0\psi_1 + b_1\psi_2 + b_2\psi_3 + b_3\psi_4 + b_4\psi_5). \quad (1.31)$$

Expand each term about x_1 (cf. the expansions used to obtain Equation (1.24)), retaining enough terms to enforce $\mathcal{O}(\Delta^4)$ accuracy [1, 7]:

$$\psi(x_1 + m\Delta) = \psi(x_1) + m\Delta \psi'(x_1) + \frac{m^2\Delta^2}{2} \psi''(x_1) + \frac{m^3\Delta^3}{6} \psi^{(3)}(x_1) + \frac{m^4\Delta^4}{24} \psi^{(4)}(x_1) + \frac{m^5\Delta^5}{120} \psi^{(5)}(x_1) + \mathcal{O}(\Delta^6). \quad (1.32)$$

Substituting Equation (1.32) into Equation (1.31) and dividing by Δ^2 gives a linear combination of derivatives at x_1 .

Choosing coefficients to cancel lower-order terms in the Taylor expansion (up to degree five) gives

$$\begin{aligned} \sum_{m=-1}^4 b_m &= 0, & \sum_{m=-1}^4 m b_m &= 0, & \sum_{m=-1}^4 m^2 b_m &= 2, \\ \sum_{m=-1}^4 m^3 b_m &= 0, & \sum_{m=-1}^4 m^4 b_m &= 0, & \sum_{m=-1}^4 m^5 b_m &= 0. \end{aligned} \quad (1.33)$$

Solving Equation (1.33) yields

$$(b_{-1}, b_0, b_1, b_2, b_3, b_4) = \left(\frac{5}{6}, -\frac{5}{4}, -\frac{1}{3}, \frac{7}{6}, -\frac{1}{2}, \frac{1}{12}\right), \quad (1.34)$$

and, after applying $\psi_0 = 0$, the implemented boundary closure is

$$\psi''(x_1) \approx \frac{-\frac{5}{4}\psi_1 - \frac{1}{3}\psi_2 + \frac{7}{6}\psi_3 - \frac{1}{2}\psi_4 + \frac{1}{12}\psi_5}{\Delta^2} + \mathcal{O}(\Delta^4). \quad (1.35)$$

Second interior node x_2 (centered stencil with known boundary). At x_2 , the standard centered five-point stencil Equation (1.26) involves ψ_0 , but $\psi_0 = 0$ is known exactly. Thus

$$\psi''(x_2) = \frac{-\psi_4 + 16\psi_3 - 30\psi_2 + 16\psi_1 - \psi_0}{12\Delta^2} + \mathcal{O}(\Delta^4) = \frac{\frac{4}{3}\psi_1 - \frac{5}{2}\psi_2 + \frac{4}{3}\psi_3 - \frac{1}{12}\psi_4}{\Delta^2} + \mathcal{O}(\Delta^4). \quad (1.36)$$

Right boundary closures. By symmetry, the bottom boundary rows use the same construction as the top boundary rows: at x_{N-1} ,

$$\psi''(x_{N-1}) \approx \frac{-\frac{1}{12}\psi_{N-3} + \frac{4}{3}\psi_{N-2} - \frac{5}{2}\psi_{N-1} + \frac{4}{3}\psi_N}{\Delta^2}, \quad (1.37)$$

and at x_N ,

$$\psi''(x_N) \approx \frac{\frac{1}{12}\psi_{N-4} - \frac{1}{2}\psi_{N-3} + \frac{7}{6}\psi_{N-2} - \frac{1}{3}\psi_{N-1} - \frac{5}{4}\psi_N}{\Delta^2}. \quad (1.38)$$

First-derivative boundary closures (for general $B(x)$). The same boundary-row logic is required for the first-derivative term $B(x)\psi'(x)$ in Equation (1.15). The fourth-order one-sided/shifted closures are

$$\psi'(x_1) \approx \frac{-\frac{1}{5}\psi_0 - \frac{13}{12}\psi_1 + 2\psi_2 - \psi_3 + \frac{1}{3}\psi_4 - \frac{1}{20}\psi_5}{\Delta} + \mathcal{O}(\Delta^4), \quad (1.39)$$

$$\psi'(x_2) \approx \frac{-\frac{2}{3}\psi_1 + \frac{2}{3}\psi_3 - \frac{1}{12}\psi_4}{\Delta} + \mathcal{O}(\Delta^4), \quad (1.40)$$

$$\psi'(x_{N-1}) \approx \frac{\frac{1}{12}\psi_{N-3} - \frac{2}{3}\psi_{N-2} + \frac{2}{3}\psi_N}{\Delta} + \mathcal{O}(\Delta^4), \quad (1.41)$$

$$\psi'(x_N) \approx \frac{\frac{1}{20}\psi_{N-4} - \frac{1}{3}\psi_{N-3} + \psi_{N-2} - 2\psi_{N-1} + \frac{13}{12}\psi_N + \frac{1}{5}\psi_{N+1}}{\Delta} + \mathcal{O}(\Delta^4), \quad (1.42)$$

with $\psi_0 = \psi_{N+1} = 0$ applied in the assembled boundary rows.

How this enters the operator. For Schrödinger Hamiltonians $H = -\frac{1}{2}\partial_{xx} + V(x)$ we have $A \equiv -\frac{1}{2}$, $B \equiv 0$, $C \equiv V$, so the boundary closures above directly define the first/last two rows of the discrete Laplacian (scaled by A/Δ^2) while the potential contributes the diagonal term $V(x_i)$ in every row. The interior rows $i = 3, \dots, N-2$ use the centered stencil Equation (1.26). This replacement strategy preserves global fourth-order accuracy while keeping the operator banded and avoiding explicit ghost variables.

Remark (general $B(x)$). For nonzero $B(x)$, the operator uses the explicit first-derivative boundary closures given above together with the second-derivative closures.

1.5 Performance Analysis and Computational Efficiency

1.5.1 Sparse Matrix Storage and Computational Scaling Analysis

The finite difference matrices derived in this chapter have a crucial property: they are *sparse*, meaning most entries are zero. Understanding sparse matrix storage is essential for efficient implementation of these methods.

Dense vs. Sparse Matrix Storage. A standard (dense) matrix storage scheme stores an $N \times N$ matrix as a list of N lists, each of length N . Every matrix component is explicitly represented, requiring exactly N^2 memory locations regardless of how many entries are actually nonzero.

In contrast, a sparse matrix storage scheme (such as COO or CSR format) stores only the nonzero elements. For each nonzero component, three values are stored: its row index, column index, and data value. This approach is advantageous when the matrix is actually sparse, but becomes inefficient for dense matrices. For example, if all N^2 components are nonzero, sparse storage requires $3N^2$ values; three times more than dense storage.

However, when matrices are truly sparse, the savings are dramatic. Consider an $N \times N$ matrix with only one nonzero element: sparse storage requires exactly 3 values, compared to N^2 for dense storage. This advantage grows rapidly with N .

Application to Finite Difference Matrices. For our finite difference matrices:

- **Second-order (tridiagonal):** $3N - 2$ nonzero entries $\Rightarrow 3(3N - 2) = 9N - 6$ values in sparse storage
- **Fourth-order (pentadiagonal):** $5N - 4$ nonzero entries $\Rightarrow 3(5N - 4) = 15N - 12$ values in sparse storage

Compared to dense storage (N^2 values), sparse storage becomes advantageous even for modest N . For $N \geq 10$, sparse storage is always preferable, and the advantage grows as N increases: sparse storage scales as $\mathcal{O}(N)$ while dense storage scales as $\mathcal{O}(N^2)$.

Time and Memory Implications. Any matrix operation: addition, multiplication, inversion, or eigenvalue computation, requires creating the matrix, storing its values in memory, and accessing them during computation. The computational cost therefore scales with the number of stored values. For sparse matrices, this keeps arithmetic work proportional to the number of nonzeros, so

- Matrix-vector products scale as $\mathcal{O}(N)$ instead of $\mathcal{O}(N^2)$
- Memory requirements scale linearly with problem size
- Iterative eigensolvers become highly efficient for finding the lowest few eigenpairs [4, 10]

Equally important, the storage format determines whether the problem fits in memory at all. Sparse storage delays the point at which a calculation exhausts the available RAM; when N is large enough for the dense representation to exceed memory, the dense computation fails outright, while the sparse version still fits and finishes.

This sparse structure is fundamental to making finite difference methods practical for large-scale problems.

1.5.2 Accuracy and Convergence Behavior

The accuracy of finite difference methods depends on two key factors: the grid spacing Δ and the choice of computational domain $[x_{\text{left}}, x_{\text{right}}]$.

Grid Spacing and Convergence Rates. The truncation error in finite difference approximations determines how rapidly the numerical solution approaches the exact solution as $\Delta \rightarrow 0$. As derived in the previous sections, the well-known convergence rates are [1, 2]:

- **Second-order method:** Eigenvalue errors scale as $\mathcal{O}(\Delta^2)$
- **Fourth-order method:** Eigenvalue errors scale as $\mathcal{O}(\Delta^4)$

This means that halving Δ reduces errors by a factor of 4 for second-order methods and by a factor of 16 for fourth-order methods. The fourth-order method achieves higher accuracy for a given Δ , or equivalently, achieves the same accuracy with a larger (coarser) grid.

Domain Selection. The choice of domain endpoints $[x_{\text{left}}, x_{\text{right}}]$ is problem-dependent. For bound states in quantum mechanics, the wavefunction must decay to zero at the boundaries. The domain must be large enough that the wavefunction is negligible at x_{left} and x_{right} , effectively approximating $x \rightarrow \pm\infty$ (discussed further in the quantum mechanics chapter). A well-known practical resolution rule is to use at least 10–15 grid points per smallest wavelength: $\Delta \lesssim \lambda_{\text{min}}/10$ [6].

1.5.3 Systematic Error Analysis

Our eigenvalue analysis framework enables systematic validation of the finite difference method through:

- **Multi-eigenvalue comparison:** Computing the first N eigenvalues simultaneously and comparing with analytical solutions
- **Convergence rate verification:** Confirming that numerical errors scale as $\mathcal{O}(\Delta^n)$ for n th-order methods
- **Relative vs. absolute error analysis:** Understanding how errors propagate differently for different eigenvalue levels
- **Method comparison:** Direct comparison of 2nd-order vs. 4th-order accuracy for identical problems

This systematic approach provides quantitative validation of both the theoretical convergence rates and the practical implementation accuracy.

1.5.4 Unified Implementation Architecture

The implementation uses a unified architecture that eliminates code duplication and provides a single, extensible framework for finite difference methods of any order. This unified approach offers several key advantages:

- **Extensibility:** Adding higher-order methods requires only updating coefficient arrays
- **Maintainability:** Single codebase eliminates duplication and reduces bugs
- **Consistency:** All methods use identical matrix assembly logic
- **Performance:** Unified approach optimizes memory usage and computation

The unified implementation centralizes finite difference coefficients through a single coefficient management system, constructs all matrix diagonals using order-agnostic functions, and handles boundary corrections through unified, order-aware functions. This architecture ensures that all finite difference methods share the same robust, well-tested implementation while maintaining flexibility for different orders and boundary conditions.

1.5.5 The Accuracy-Efficiency Trade-off

In numerical methods, improved accuracy usually requires additional computational effort. In this setting, second- and fourth-order discretizations follow that same trade-off, but the fourth-order method typically delivers more accuracy per degree of freedom and can therefore be preferable at fixed error targets.

Computational Cost Scaling. As established in §1.5.1, the computational cost scales with the number of grid points N . From the grid spacing relationship $\Delta = (x_{\text{right}} - x_{\text{left}})/(N + 1)$, achieving a finer grid requires increasing N :

- Halving Δ requires doubling N
- For second-order: storage scales as $9N - 6 \approx 9N$ (linear in N)

- For fourth-order: storage scales as $15N - 12 \approx 15N$ (also linear in N)
- Matrix operations scale as $\mathcal{O}(N)$ for sparse matrices

Thus, computational time grows approximately linearly with N , or equivalently, inversely with Δ : finer grids cost more time.

Accuracy Scaling. As established in §1.5.2, accuracy improves as Δ decreases:

- Second-order: error $\sim C_2 \Delta^2$
- Fourth-order: error $\sim C_4 \Delta^4$

To achieve a target error ϵ , the required grid spacing is:

$$\text{Second-order: } \Delta_2 \sim \sqrt{\epsilon/C_2} \tag{1.43}$$

$$\text{Fourth-order: } \Delta_4 \sim (\epsilon/C_4)^{1/4} \tag{1.44}$$

The Key Trade-off. Higher accuracy demands smaller Δ , which requires larger N , which increases computational cost. The fourth-order method achieves better accuracy for a given Δ (or equivalently, uses a larger Δ to achieve a given accuracy), potentially reducing N and computational time.

For high-accuracy applications, the fourth-order method can be significantly more efficient: achieving error ϵ requires $\Delta_4 \sim \epsilon^{1/4}$ compared to $\Delta_2 \sim \epsilon^{1/2}$. For $\epsilon = 10^{-8}$, this gives $\Delta_4 \sim 10^{-2}$ versus $\Delta_2 \sim 10^{-4}$ —a factor of 100 coarser grid spacing, which translates to roughly 100 times fewer grid points and 100 times faster computation.

However, the fourth-order method has a larger stencil (5-point vs. 3-point), making each matrix entry more expensive to compute. The optimal choice depends on required accuracy and problem size. For modest accuracy requirements, second-order methods may suffice; for stricter error targets, fourth-order methods are often advantageous [2].

Chapter 2

Applications in Quantum Mechanics

2.1 Introduction to Quantum Eigenvalue Problems

The finite difference method is particularly well-suited for solving the time-independent Schrödinger equation, which is a special case of the general eigenvalue problem Equation (1.1) [3, 11].

2.1.1 The Time-Independent Schrödinger Equation

In one dimension, a definite energy state satisfies the well-known equation [3]

$$\hat{H}\psi = -\frac{\hbar^2}{2m}\psi''(x) + V(x)\psi(x) = E\psi(x). \quad (2.1)$$

Normalizability $\int |\psi|^2 dx = 1$ implies $\psi(\pm\infty) = 0$. For confining V , the spectrum is discrete and indexed by $k = 0, 1, 2, \dots$, as is well known [11].

This is a special case of Equation (1.1) in which the operator coefficients of Equation (1.2) satisfy:

$$A(x) = -\frac{\hbar^2}{2m} \quad (2.2)$$

$$B(x) = 0 \quad (2.3)$$

$$C(x) = V(x) \quad (2.4)$$

$$\lambda = E \quad (2.5)$$

The boundary conditions depend on the specific quantum system being studied.

2.1.2 General Nondimensionalization Procedure

The dimensional Schrödinger equation Equation (2.1) contains physical constants $\hbar$ and m that complicate numerical analysis. The well-known technique of nondimensionalization transforms this into a mathematically cleaner form while preserving the essential physics [3].

The specific choice of characteristic length scale x_* depends on the physical system being studied. However, regardless of this choice, we adopt a universal relationship between the length and energy scales that simplifies the Schrödinger equation.

We introduce a characteristic length scale x_* (to be specified for each system) and define the characteristic energy scale through:

$$E_* = \frac{\hbar^2}{2mx_*^2}. \quad (2.6)$$

This particular relationship ensures that the kinetic energy term has a simple form in dimensionless units.

Define dimensionless variables:

$$\tilde{x} = \frac{x}{x_*}, \quad (2.7)$$

$$\tilde{E} = \frac{E}{E_*} = \frac{2mEx_*^2}{\hbar^2}, \quad (2.8)$$

$$\tilde{V}(\tilde{x}) = \frac{V(x_*\tilde{x})}{E_*} = \frac{2mx_*^2}{\hbar^2} V(x_*\tilde{x}). \quad (2.9)$$

Substituting into Equation (2.1) and dividing by E_* :

$$-\frac{\hbar^2}{2mE_*} \frac{d^2\psi}{dx^2} + \frac{V(x)}{E_*} \psi = \frac{E}{E_*} \psi, \quad (2.10)$$

$$-\frac{\hbar^2}{2m} \cdot \frac{2mx_*^2}{\hbar^2} \cdot \frac{1}{x_*^2} \frac{d^2\psi}{d\tilde{x}^2} + \tilde{V}(\tilde{x})\psi = \tilde{E}\psi, \quad (2.11)$$

$$-\frac{d^2\psi}{d\tilde{x}^2} + \tilde{V}(\tilde{x})\psi = \tilde{E}\psi. \quad (2.12)$$

This yields the dimensionless Schrödinger equation:

$$-\frac{d^2\psi}{d\tilde{x}^2} + \tilde{V}(\tilde{x})\psi = \tilde{E}\psi. \quad (2.13)$$

The dimensionless form eliminates physical constants and provides a universal framework for numerical solution. In the following sections, we apply this framework to two fundamental quantum systems: the infinite square well and the quantum harmonic oscillator.

2.2 The Infinite Square Well

The infinite square well (ISW) represents one of the fundamental quantum mechanical systems, providing an excellent test case for finite difference methods due to its well-known simple analytical solution and clear boundary conditions [3].

2.2.1 Problem Statement and Nondimensionalization

The ISW consists of a particle of mass m confined to a one-dimensional box of width L . The potential energy is:

$$V(x) = \begin{cases} 0 & \text{if } 0 < x < L \\ \infty & \text{if } x \leq 0 \text{ or } x \geq L \end{cases} \quad (2.14)$$

The infinite potential barriers create perfect confinement. The wavefunction must vanish at the boundaries:

$$\psi(0) = 0, \quad \psi(L) = 0. \quad (2.15)$$

Following the general nondimensionalization procedure from §2.1.2, we choose the characteristic length scale as the well width:

$$x_* = L. \quad (2.16)$$

This choice gives the characteristic energy scale:

$$E_* = \frac{\hbar^2}{2mL^2}. \quad (2.17)$$

With dimensionless variables $\tilde{x} = x/L$, $\tilde{E} = E/E_*$, and $\tilde{V}(\tilde{x}) = V(Lx\tilde{x})/E_*$, the potential becomes:

$$\tilde{V}(\tilde{x}) = \begin{cases} 0 & \text{if } 0 < \tilde{x} < 1 \\ \infty & \text{otherwise} \end{cases} \quad (2.18)$$

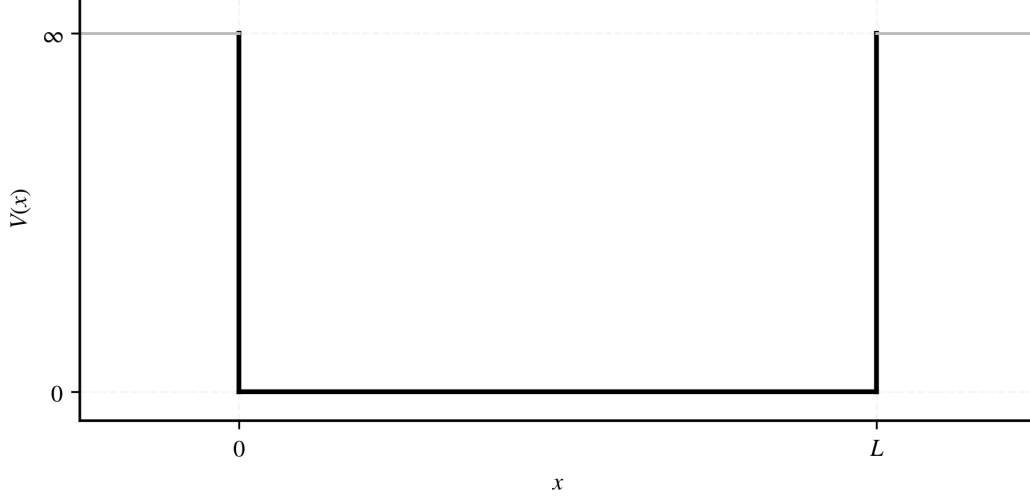


Figure 2.1: Schematic of the infinite square well potential [12].

The dimensionless Schrödinger equation inside the well ($0 < \tilde{x} < 1$) is:

$$-\frac{d^2\psi}{d\tilde{x}^2} = \tilde{E}\psi, \quad (2.19)$$

with boundary conditions:

$$\psi(0) = 0, \quad \psi(1) = 0. \quad (2.20)$$

2.2.2 Analytical Solution

We now solve the dimensionless eigenvalue problem Equation (2.19) analytically.

General Solution via Ansatz Method. The differential equation $-\psi'' = \tilde{E}\psi$ is a second-order linear ODE with constant coefficients. We seek solutions of the form $\psi(\tilde{x}) = e^{r\tilde{x}}$. Substituting:

$$-r^2 e^{r\tilde{x}} = \tilde{E} e^{r\tilde{x}} \Rightarrow r^2 = -\tilde{E}. \quad (2.21)$$

For bound states, we require $\tilde{E} > 0$, giving $r = \pm i\sqrt{\tilde{E}}$. Define $\tilde{k} = \sqrt{\tilde{E}}$ (real and positive). The general solution is a linear combination:

$$\psi(\tilde{x}) = Ae^{i\tilde{k}\tilde{x}} + Be^{-i\tilde{k}\tilde{x}} = C \sin(\tilde{k}\tilde{x}) + D \cos(\tilde{k}\tilde{x}), \quad (2.22)$$

where we've converted to the real form using Euler's formula. We can verify this is a solution:

$$-\frac{d^2}{d\tilde{x}^2} [C \sin(\tilde{k}\tilde{x}) + D \cos(\tilde{k}\tilde{x})] = \tilde{k}^2 [C \sin(\tilde{k}\tilde{x}) + D \cos(\tilde{k}\tilde{x})] = \tilde{E}\psi, \quad (2.23)$$

confirming the ansatz.

Applying Boundary Conditions and Quantization. The boundary condition at $\tilde{x} = 0$ requires:

$$\psi(0) = C \sin(0) + D \cos(0) = D = 0. \quad (2.24)$$

Thus $D = 0$, and the solution reduces to:

$$\psi(\tilde{x}) = C \sin(\tilde{k}\tilde{x}). \quad (2.25)$$

The boundary condition at $\tilde{x} = 1$ requires:

$$\psi(1) = C \sin(\tilde{k}) = 0. \quad (2.26)$$

For a nontrivial solution ($C \neq 0$), we need $\sin(\tilde{k}) = 0$, which occurs when:

$$\tilde{k} = n\pi, \quad n = 1, 2, 3, \dots \quad (2.27)$$

Note that $n = 0$ gives $\psi \equiv 0$ (not normalizable), and negative n values give the same solutions as positive n (since $\sin(-x) = -\sin(x)$, differing only by an overall sign). Thus the allowed energy eigenvalues are:

$$\tilde{E}_n = \tilde{k}_n^2 = n^2\pi^2, \quad n = 1, 2, 3, \dots \quad (2.28)$$

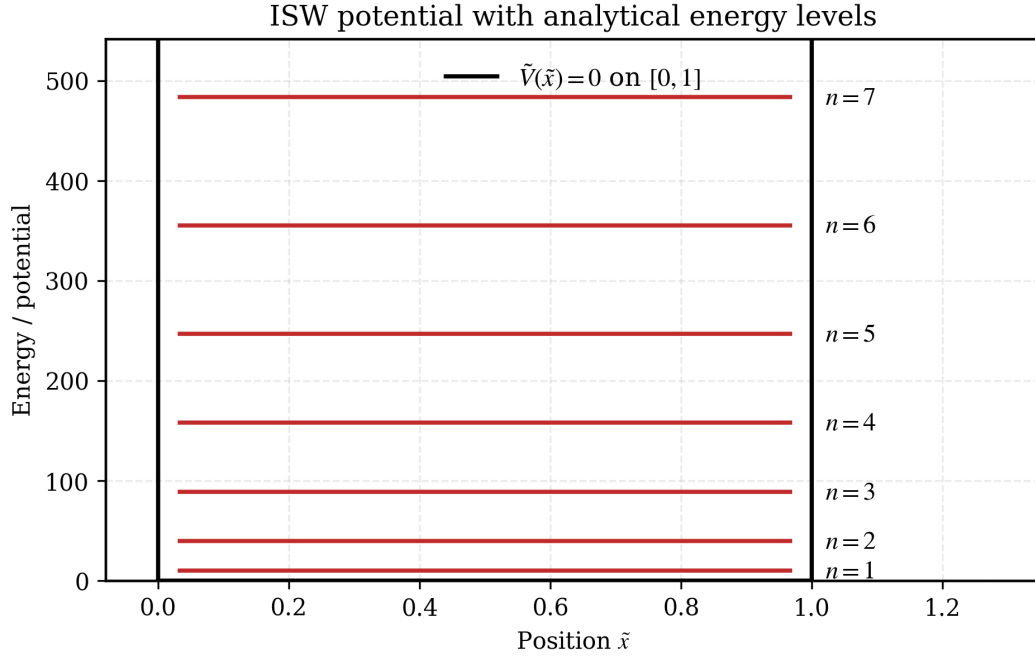


Figure 2.2: Dimensionless ISW potential and analytical energy levels, shown in the same visual style as the QSHO spectrum. The well is defined on $\tilde{x} \in [0, 1]$ with $\tilde{V}(\tilde{x}) = 0$ inside and walls at $\tilde{x} = 0, 1$; levels $\tilde{E}_n = n^2\pi^2$ for $n = 1, \dots, 7$ are plotted as horizontal lines. Detailed plotting specifications are listed in Section A.5.

Normalization. The unnormalized eigenfunctions are $\psi_n(\tilde{x}) = C_n \sin(n\pi\tilde{x})$. To normalize, we require:

$$\int_0^1 |\psi_n(\tilde{x})|^2 d\tilde{x} = 1. \quad (2.29)$$

Computing the integral:

$$\int_0^1 C_n^2 \sin^2(n\pi\tilde{x}) d\tilde{x} = C_n^2 \int_0^1 \frac{1 - \cos(2n\pi\tilde{x})}{2} d\tilde{x} \quad (2.30)$$

$$= C_n^2 \left[\frac{\tilde{x}}{2} - \frac{\sin(2n\pi\tilde{x})}{4n\pi} \right]_0^1 \quad (2.31)$$

$$= C_n^2 \cdot \frac{1}{2} = 1. \quad (2.32)$$

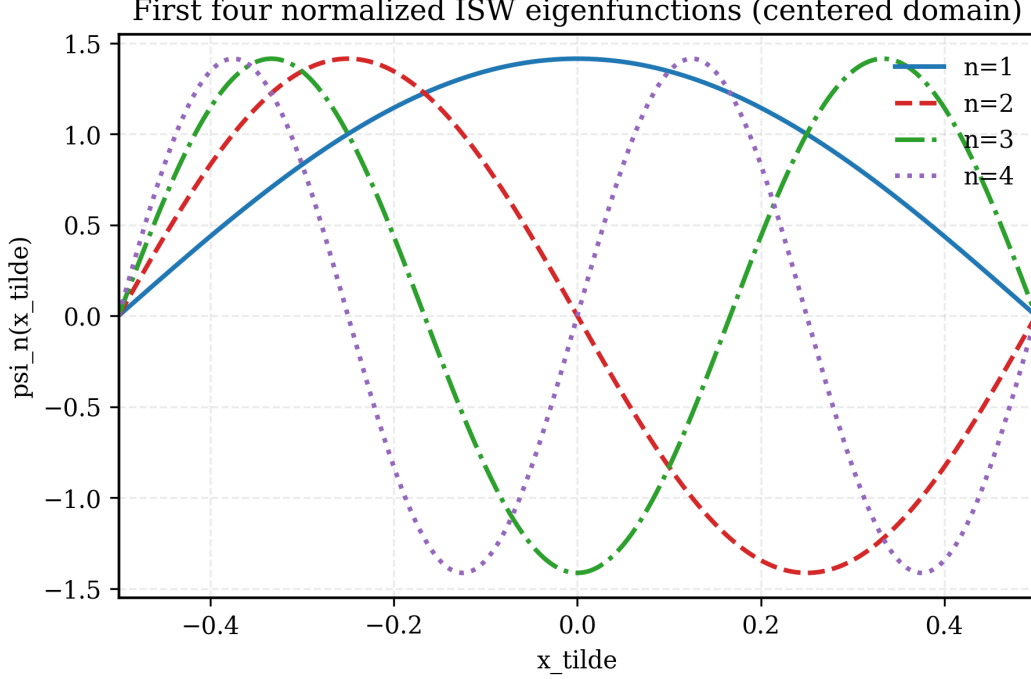


Figure 2.3: First four normalized analytical ISW eigenfunctions on a centered dimensionless domain. The plotted modes are $\psi_n(\tilde{x}) = \sqrt{2} \sin(n\pi(\tilde{x} + \frac{1}{2}))$ for $n = 1, 2, 3, 4$. Detailed plotting specifications are listed in Section A.5.

Therefore $C_n = \sqrt{2}$, and the normalized eigenfunctions are:

$$\psi_n(\tilde{x}) = \sqrt{2} \sin(n\pi\tilde{x}), \quad n = 1, 2, 3, \dots \quad (2.33)$$

The complete solution to the dimensionless infinite square well is:

$$\text{Eigenvalues: } \tilde{E}_n = n^2 \pi^2, \quad n = 1, 2, 3, \dots \quad (2.34)$$

$$\text{Eigenfunctions: } \psi_n(\tilde{x}) = \sqrt{2} \sin(n\pi\tilde{x}). \quad (2.35)$$

Because all subsequent numerical experiments use the nondimensional formulation, we will quote results in terms of Equations (2.28) and (2.33) without converting back to dimensional units.

2.2.3 Spectral Features and Discretization Implications

Figure 2.2 shows that ISW energy levels are discrete and scale quadratically with n , with no zero-point energy. It also reinforces why the ISW is a strong benchmark for finite-difference methods: the potential and boundary conditions are simple, and analytical eigenpairs are available for direct verification.

Each n th ISW eigenfunction has $n - 1$ interior nodes, so the wavelength is $\lambda_n = 2/n$ in dimensionless units; resolving higher-energy states therefore requires proportionally finer grids. Combining this with the rule-of-thumb from §1.5.2 implies $N \gtrsim 10n$ interior points to resolve the n th mode accurately. The oscillations also concentrate away from the boundaries, making uniform grids adequate provided the endpoints align with the Dirichlet walls. Numerical validation of the ISW spectrum (second- vs. fourth-order against the analytical eigenvalues) is presented in Chapter 3; see Figure 3.5.

2.3 The Quantum Simple Harmonic Oscillator

The quantum simple harmonic oscillator (QSHO) is another fundamental system in quantum mechanics, representing particles in parabolic potentials. Unlike the ISW, the QSHO features an infinite domain and a

nontrivial potential, making it a complementary test case for finite difference methods [3, 11].

2.3.1 Problem Statement and Nondimensionalization

The QSHO describes a particle of mass m subject to a harmonic restoring force with angular frequency ω . The potential energy is:

$$V(x) = \frac{1}{2}m\omega^2x^2. \quad (2.36)$$

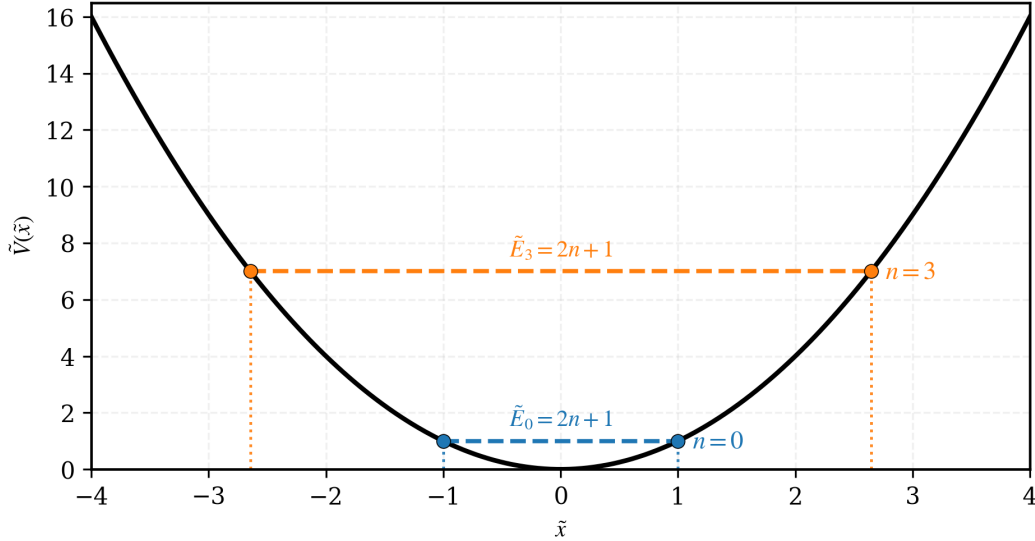


Figure 2.4: Dimensionless QSHO potential $\tilde{V}(\tilde{x}) = \tilde{x}^2$ with classical turning points illustrated for two analytical levels ($n = 0$ and $n = 3$). Horizontal dashed lines mark $\tilde{E}_n = 2n + 1$, and vertical dotted lines indicate $\tilde{x}_t = \pm\sqrt{\tilde{E}_n}$. Detailed plotting specifications are listed in Section A.5.

In dimensionless form, the classical turning points follow from $\tilde{V}(\tilde{x}) = \tilde{x}^2 = \tilde{E}_n = 2n + 1$, giving $\tilde{x}_t = \pm\sqrt{2n + 1}$. Although this is a classical construction, the corresponding quantum states also spread spatially as n increases, so higher excited states typically require larger computational domains and finer grids for accurate resolution.

For bound states, the wavefunction must be normalizable, requiring $\psi(x) \rightarrow 0$ as $x \rightarrow \pm\infty$.

Following the general nondimensionalization procedure, we choose the characteristic length scale as the natural oscillator length:

$$x_* = \sqrt{\frac{\hbar}{m\omega}}. \quad (2.37)$$

This gives the characteristic energy scale from the general relationship $E_* = \hbar^2/(2mx_*^2)$:

$$E_* = \frac{\hbar^2}{2m} \cdot \frac{m\omega}{\hbar} = \frac{\hbar\omega}{2}. \quad (2.38)$$

The dimensionless potential becomes:

$$\tilde{V}(\tilde{x}) = \frac{V(x_*\tilde{x})}{E_*} = \frac{1}{E_*} \cdot \frac{1}{2}m\omega^2(x_*\tilde{x})^2 \quad (2.39)$$

$$= \frac{2}{\hbar\omega} \cdot \frac{1}{2}m\omega^2 \cdot \frac{\hbar}{m\omega} \cdot \tilde{x}^2 = \tilde{x}^2. \quad (2.40)$$

The dimensionless Schrödinger equation is:

$$-\frac{d^2\psi}{d\tilde{x}^2} + \tilde{x}^2\psi = \tilde{E}\psi. \quad (2.41)$$

2.3.2 Analytical Solution

The full analytical derivation of QSHO eigenstates involves power series methods and Hermite polynomials, which is lengthy and beyond our scope. We present the well-known final results here; the complete derivation can be found in standard quantum mechanics texts [3, 11].

The energy eigenvalues in dimensionless units are:

$$\tilde{E}_n = 2n + 1, \quad n = 0, 1, 2, \dots \quad (2.42)$$

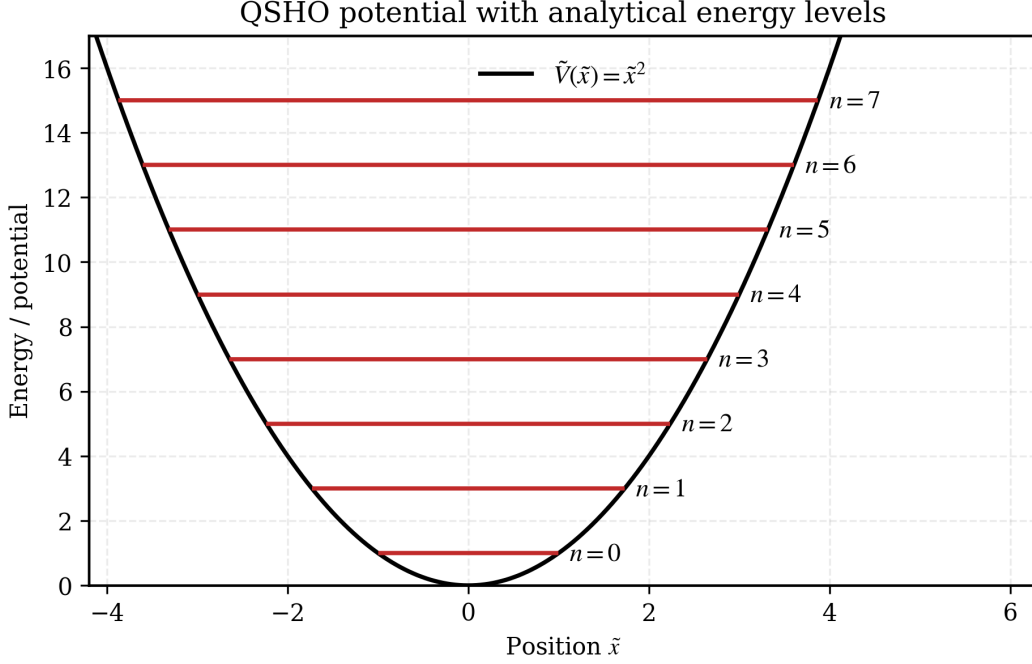


Figure 2.5: Dimensionless QSHO potential and analytical bound-state energy levels. The potential is $\tilde{V}(\tilde{x}) = \tilde{x}^2$, and the plotted levels are $\tilde{E}_n = 2n + 1$ for $n = 0, \dots, 7$. Detailed plotting specifications are listed in Section A.5.

Note the zero-point energy: even the ground state ($n = 0$) has $\tilde{E}_0 = 1$. In dimensional units:

$$E_n = \tilde{E}_n \cdot E_* = (2n + 1) \frac{\hbar\omega}{2} = \left(n + \frac{1}{2}\right) \hbar\omega. \quad (2.43)$$

The normalized eigenfunctions are:

$$\psi_n(\tilde{x}) = \frac{1}{\sqrt{2^n n!}} \left(\frac{1}{\pi}\right)^{1/4} e^{-\tilde{x}^2/2} H_n(\tilde{x}), \quad (2.44)$$

where $H_n(\tilde{x})$ are the Hermite polynomials. The first few are listed in Table 2.1.

These solutions have several important properties. The energy levels are equally spaced with $\Delta\tilde{E} = 2$ (equivalently $\Delta E = \hbar\omega$ in dimensional units), and the n th eigenfunction has exactly n nodes. All states decay to zero as $\tilde{x} \rightarrow \pm\infty$ due to the Gaussian envelope $e^{-\tilde{x}^2/2}$, which ensures normalizability. As n increases, the polynomial factor adds more oscillations and the wavefunction support spreads over a wider spatial region. Figure 2.6 visualizes the first few modes and shows how both the spatial extent and the nodal structure evolve with n .

For numerical implementation using finite difference methods, we must truncate the infinite domain to a finite computational interval $[\tilde{x}_{\text{left}}, \tilde{x}_{\text{right}}]$ chosen so that the wavefunction is negligible at the boundaries. Typical choices are $\tilde{x}_{\text{left}} = -8$ and $\tilde{x}_{\text{right}} = 8$ for the lowest few eigenstates, providing a rigorous test of both the finite difference accuracy and the domain truncation approximation.

Table 2.1: First four Hermite polynomials used in the QSHO eigenfunctions.

n	$H_n(\tilde{x})$
0	1
1	$2\tilde{x}$
2	$4\tilde{x}^2 - 2$
3	$8\tilde{x}^3 - 12\tilde{x}$

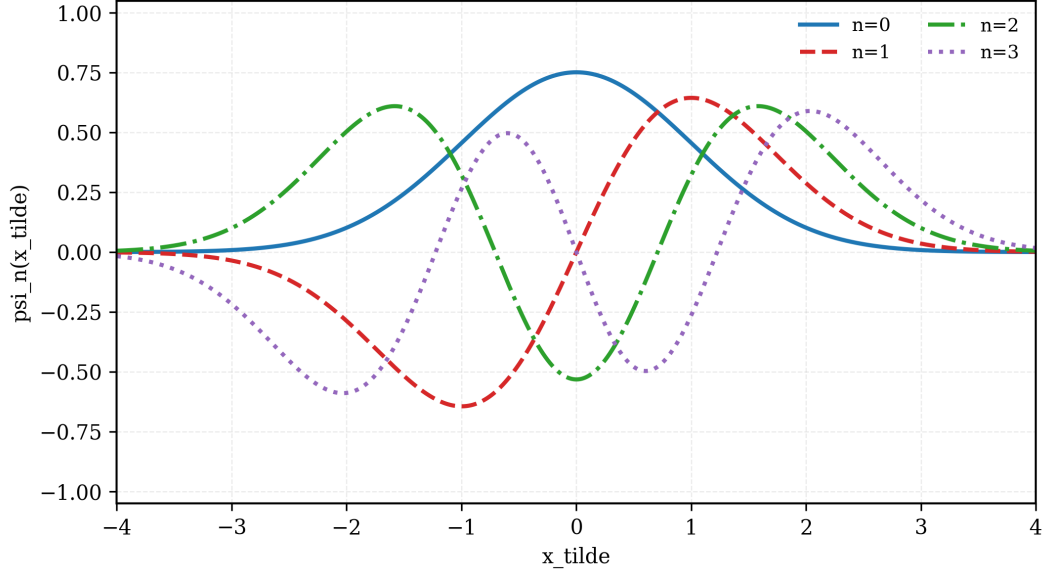


Figure 2.6: First four normalized analytical QSHO eigenfunctions in dimensionless units, centered at $\tilde{x} = 0$. The plotted modes are $\psi_n(\tilde{x}) = \frac{1}{\sqrt{2^n n! \sqrt{\pi}}} H_n(\tilde{x}) e^{-\tilde{x}^2/2}$ for $n = 0, 1, 2, 3$. Detailed plotting specifications are listed in Section A.5.

2.3.3 Spectral Features and Discretization Implications

Within a fixed truncated domain, the node spacing near the origin scales approximately as $1/\sqrt{n}$, so the resolution rule from §1.5.2 suggests $N \gtrsim 10\sqrt{n}$ interior points in each half-interval to maintain fourth-order accuracy for the n th state. Figure 2.6 highlights the corresponding increase in nodal complexity with n . The quantitative impact of domain truncation on QSHO eigenvalue accuracy is shown in Chapter 3; see Figure 3.7.

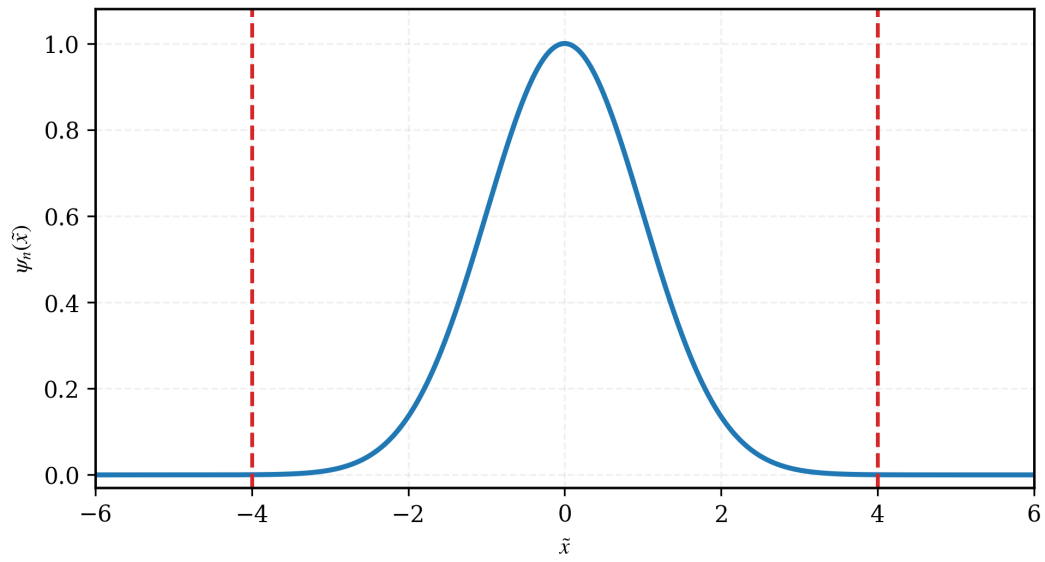


Figure 2.7: Domain truncation schematic for the QSHO: Gaussian decay of $\psi_n(\tilde{x})$ [13].

Chapter 3

Results and Analysis

This chapter presents the numerical evidence for the finite-difference framework by analyzing convergence, spectral accuracy, domain truncation sensitivity, runtime scaling, and wavefunction fidelity. Throughout these studies, resolution choices are interpreted in a system-dependent way using the practical rule $\Delta \lesssim \lambda_{\min}/10$, where $\lambda_{\min}$ is the smallest physically relevant wavelength for the states being resolved.

3.1 Runtime and Efficiency

Timing benchmarks compare computational cost versus grid size. Here we present (i) an ISW runtime comparison between second- and fourth-order methods and (ii) a measured-to-limit QSHO runtime sweep for the fourth-order method out to the practical break point.

For ISW (Figure 3.1), fourth-order runs are consistently faster than second-order at matched N in this implementation. A linear fit of $\log(\text{runtime})$ versus $\log(N)$ yields slopes m_2 and m_4 for second- and fourth-order respectively; both are superlinear ($m > 1$) in the measured range, indicating that effective solver work grows with problem size. This result is important because stencil width alone would suggest the opposite trend; however, total wall time reflects the entire solve pipeline (matrix assembly, sparse operations, and iterative eigensolver convergence), not just per-row stencil cost. In this dataset, solver-level effects dominate the net runtime and offset the extra arithmetic associated with the wider fourth-order stencil.

The QSHO measured-to-limit run (Figure 3.2) extends substantially farther in N and shows monotone log-log growth up to the stopping criterion. A linear fit of $\log(\text{runtime})$ versus $\log(N)$ over the measured points yields a slope $m \approx 1.73$, which is steeper than the ideal sparse-operator expectation $m \approx 1$ under fixed iteration count. The measured data does not follow a single power law uniformly across the range: the curve lies below the fit at small N , above it in the mid-range (e.g., $N \sim 10^3$ – 10^4), and slightly below again at the largest N . This suggests that local scaling exponents vary with problem size, so the fitted slope $m \approx 1.73$ should be interpreted as an average scaling over the full sweep. The superlinear trend indicates that effective solver work is not constant with grid size: iteration counts, spectral conditioning near the target eigenpair, and shift-invert overhead all become increasingly relevant as N grows.

Taken together, Figures 3.1 and 3.2 support the same practical conclusion as the convergence results in Figures 3.3 and 3.4: method selection should be made at target accuracy under controlled solver settings. In other words, one should compare methods by “time to reach a prescribed error” rather than by formal stencil order or by a single runtime slope in isolation. High- N runtime behavior should therefore be interpreted as a coupled discretization–solver effect rather than stencil width alone.

3.2 Convergence Studies

This section evaluates convergence for the ISW and QSHO using the relative ground-state energy error versus grid size N on log-log axes. For each benchmark, we keep the physical/domain setup fixed and vary only N , so the plotted trend primarily reflects discretization behavior. The reference is the analytical ground-state energy, and both second- and fourth-order operators are solved on the same grids for a direct comparison.

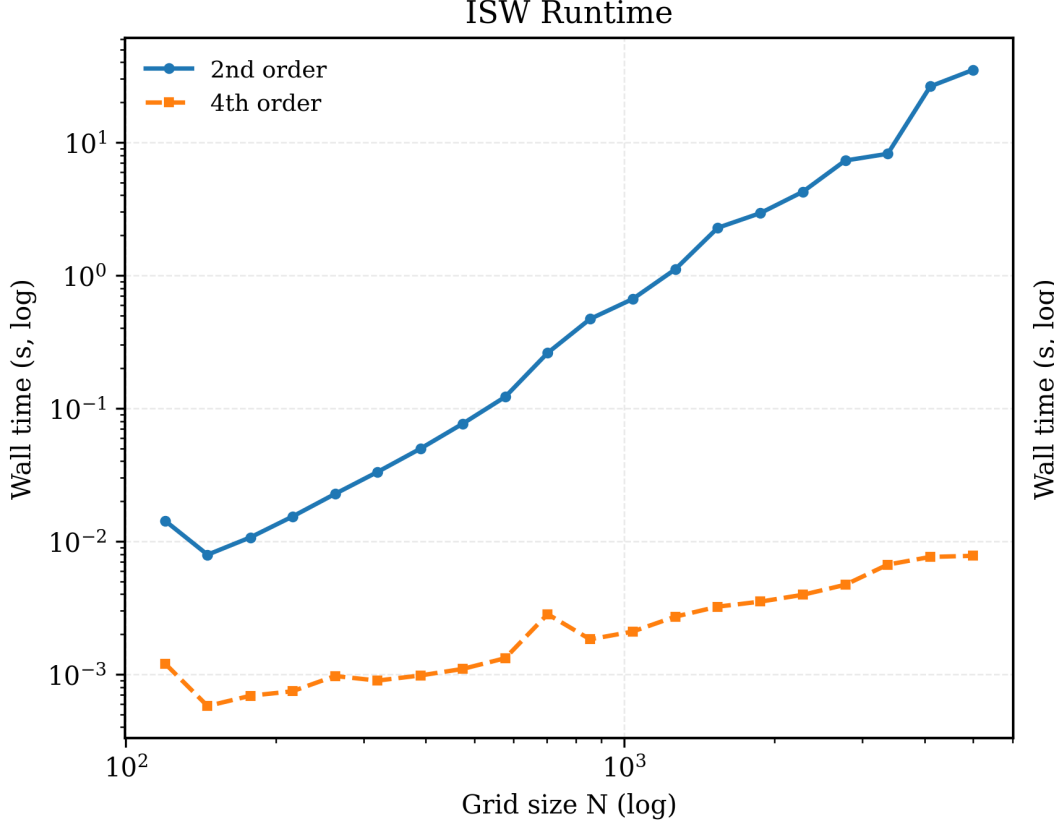


Figure 3.1: ISW runtime comparison (2nd vs 4th order) with log-log linear fit slopes. Detailed plotting specifications are listed in Section A.5.

In Figure 3.3, the second-order curve decreases smoothly with N and follows an approximately linear trend on the log-log axes, consistent with the expected $\mathcal{O}(\Delta^2)$ behavior. The fourth-order curve remains substantially lower throughout the same range and exhibits a steeper pre-asymptotic descent consistent with high-order truncation reduction. At larger N , however, the fourth-order error stops decreasing monotonically and oscillates near a very small floor. This turnover is physically plausible: once truncation error is sufficiently reduced, roundoff amplification, solver tolerance limits, and matrix-conditioning effects become comparable to (or larger than) discretization error [1, 14].

A key practical point is that the early/mid- N region is the regime most relevant to method comparison. In that regime, fourth-order delivers clearly lower error at matched N . The high- N oscillatory tail should therefore be interpreted as a regime-change diagnostic rather than evidence that fourth-order accuracy “fails”; it indicates that the dominant error mechanism has shifted away from truncation.

The same qualitative result holds for QSHO in Figure 3.4: fourth-order remains well below second-order at matched N , and the fitted second-order trend is close to the theoretical slope of -2 over the descending region. The fourth-order branch shows a steeper initial decline (close to the expected -4 behavior before asymptotic contamination), then enters a low-error plateau/oscillatory region at high resolution. For QSHO, that transition reflects three coupled effects: finite-difference truncation, eigensolver/finite-precision limits, and finite-domain truncation on $[-x_\infty, x_\infty]$.

Overall, the ISW and QSHO convergence plots support the same operational conclusion: fourth-order discretization provides substantially better accuracy per grid point in the truncation-dominated regime, while very high- N behavior must be interpreted with explicit awareness of solver and domain effects.

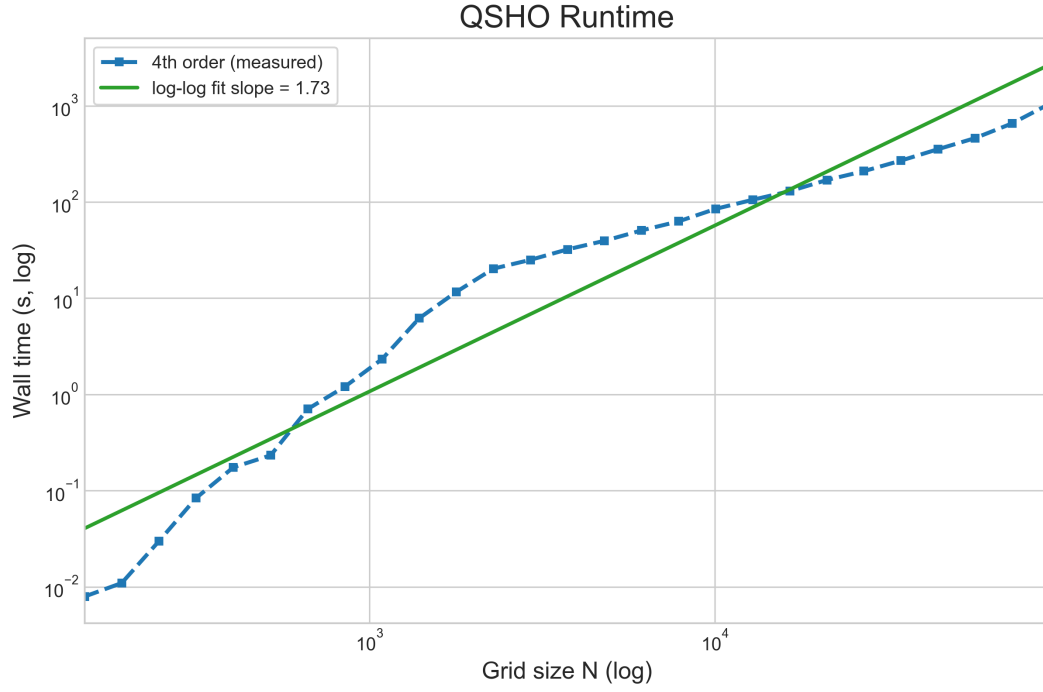


Figure 3.2: QSHO fourth-order runtime measured to practical N -break, with log-log linear fit slope. Detailed plotting specifications are listed in Section A.5.

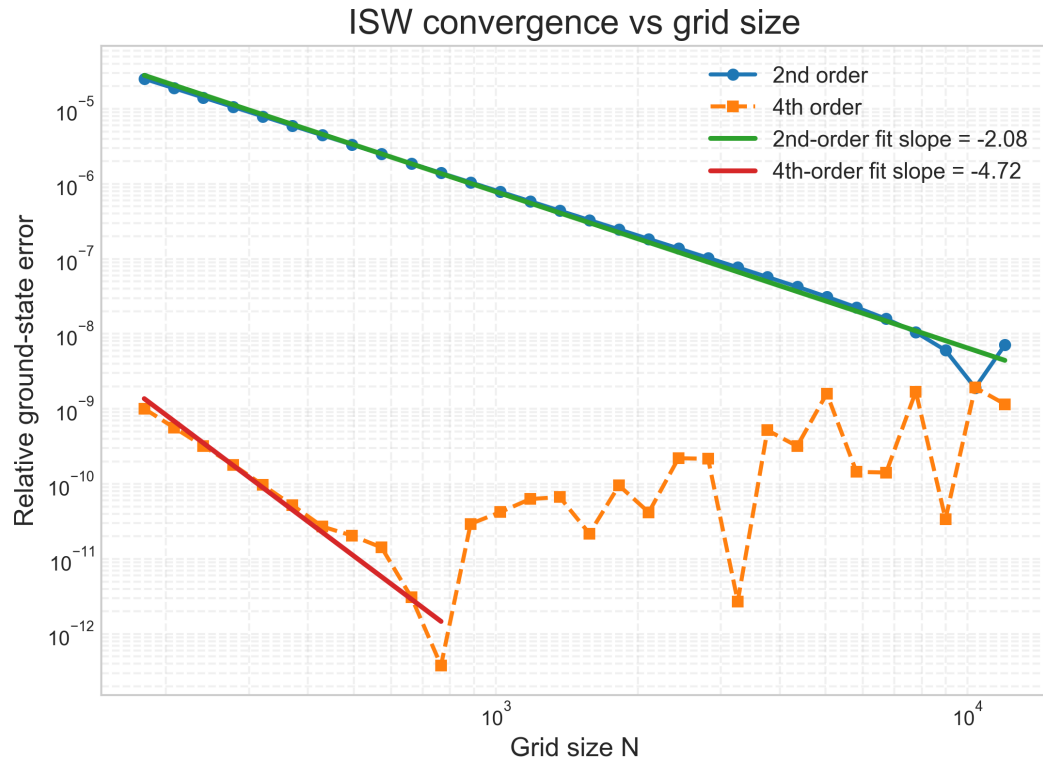


Figure 3.3: ISW convergence plot (2nd vs 4th order). Detailed plotting specifications are listed in Section A.5.

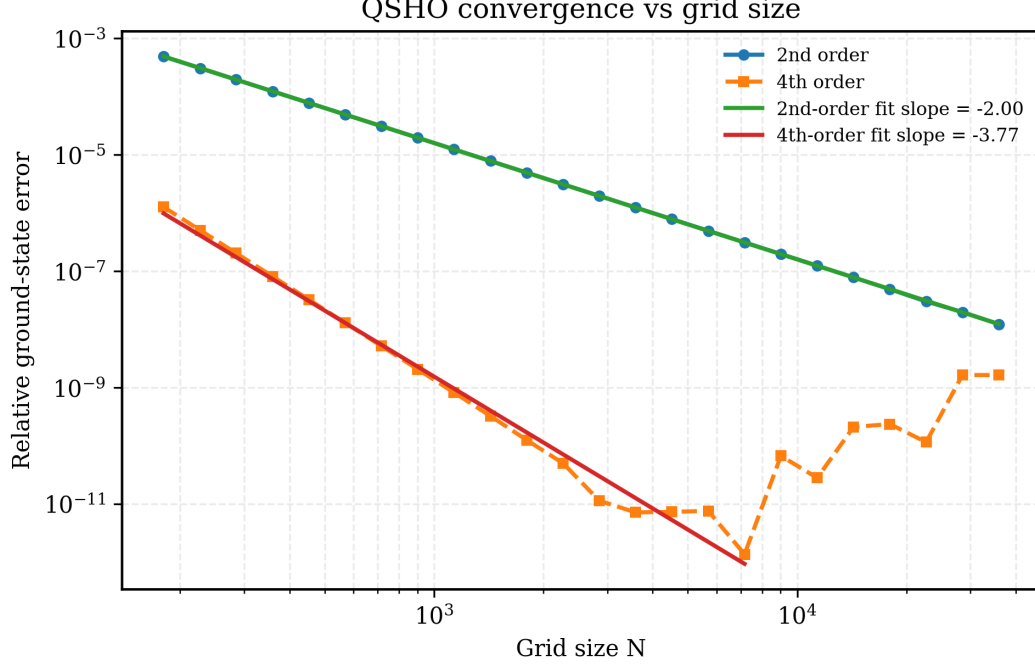


Figure 3.4: QSHO convergence plot (2nd vs 4th order). Detailed plotting specifications are listed in Section A.5.

3.3 Mode Index vs Error

To quantify spectral accuracy beyond the ground state, we compare numerical eigenvalues against analytical spectra and report mode-by-mode relative errors. This section tests the Chapter 2 prediction that higher-energy states, with denser nodal structure, are more sensitive to grid spacing; accordingly, error is expected to grow with mode index at fixed resolution.

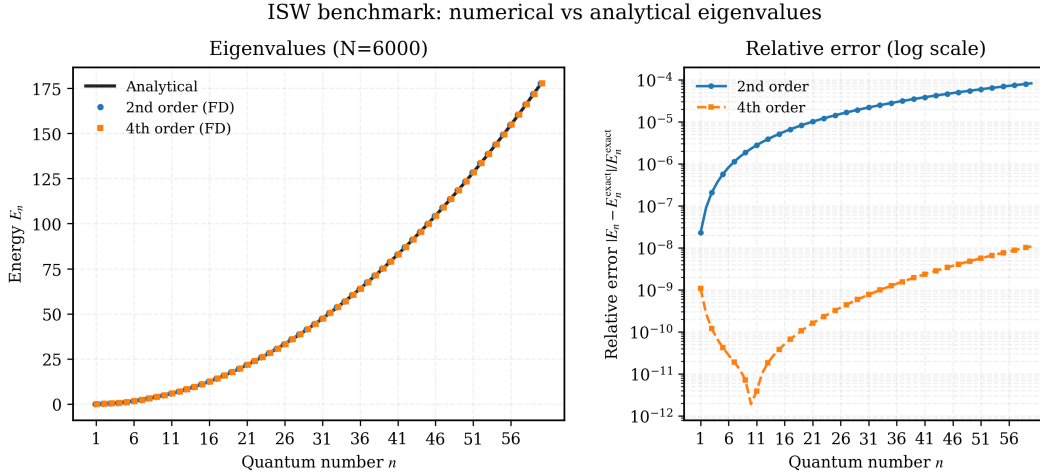


Figure 3.5: ISW eigenvalue comparison: analytical vs numerical values (left) and relative errors (right). Detailed plotting specifications are listed in Section A.5.

Figure 3.5 combines two complementary diagnostics. The left panel shows that both numerical methods track the analytical spectrum closely. The right panel (log scale) resolves the actual error magnitudes and

trends: the second-order error increases steadily with mode index, while the fourth-order error reaches a very low minimum at low/intermediate modes and then increases for higher modes. The increase at higher mode index is physically and numerically consistent: higher ISW modes oscillate on shorter length scales, so fixed- N grids undersample them first; at the same time, boundary-row closure effects and eigensolver conditioning become more influential for highly oscillatory states. Together, these effects produce the observed non-monotone fourth-order curve rather than a strictly decreasing trend across all modes [14].

The two-panel structure is important for interpretation. The left panel alone can make both methods look similarly accurate because the eigenvalue scale itself grows with mode index. The right panel, by normalizing to relative error on a log axis, reveals the method separation clearly and shows where each method begins to lose resolution efficiency. This is the primary evidence that higher-order stencils delay (but do not eliminate) high-mode degradation at fixed grid size.

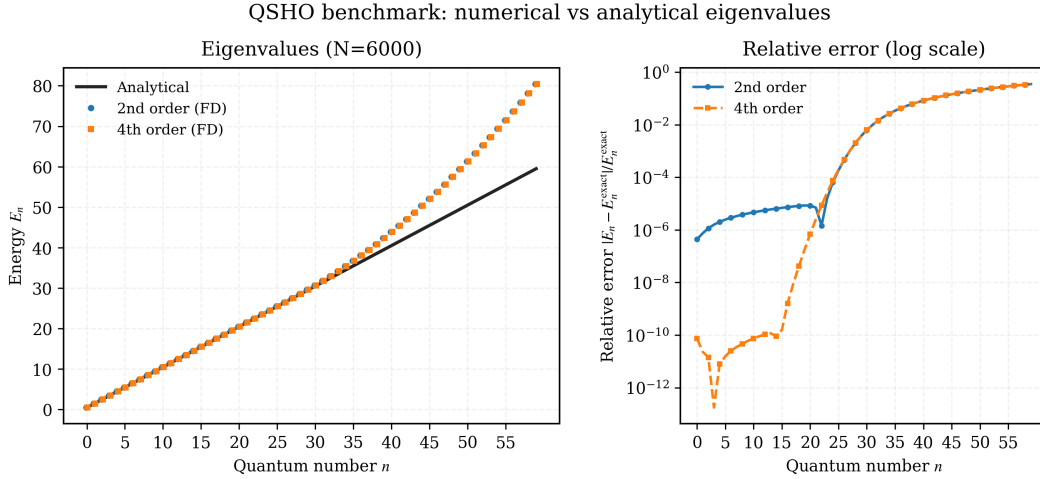


Figure 3.6: QSHO eigenvalue comparison: analytical vs numerical values (left) and relative errors (right). Detailed plotting specifications are listed in Section A.5.

3.3.1 QSHO Spectrum Validation

Figure 3.6 shows the same two-panel diagnostic used for ISW. In the left panel, both finite-difference methods reproduce the analytical QSHO spectrum closely over the plotted modes. The right panel (log scale) resolves the mode-by-mode relative error: the second-order curve grows steadily with mode index, while the fourth-order curve stays substantially lower over most of the range and reaches its smallest values at low/intermediate modes before increasing at higher n .

This trend is consistent with the Chapter 2 expectation that higher QSHO modes are harder to resolve at fixed grid spacing because they oscillate more rapidly and spread farther in space. Relative to ISW, QSHO adds an additional source of bias: the computational boundary is artificial, so high-energy states that extend farther into the tails are more sensitive to finite-domain placement. The high-mode rise in both curves therefore reflects compounded resolution pressure from discretization and domain truncation, with solver conditioning contributing at the upper end.

Comparing Figures 3.5 and 3.6 therefore isolates a useful physics-dependent distinction: mode-index growth in error is universal, but for QSHO the onset can be accelerated by domain effects even when stencil order is unchanged.

3.3.2 QSHO Domain Truncation Sensitivity

Unlike the ISW, the harmonic oscillator is posed on an infinite domain. The numerical problem therefore requires truncating to a finite interval $[-x_\infty, x_\infty]$ with homogeneous Dirichlet boundaries. The resulting error contains two contributions: discretization error (controlled by Δ and the finite difference order) and domain

truncation error (controlled by x_∞). The figure below isolates the truncation component by varying x_∞ at fixed resolution.

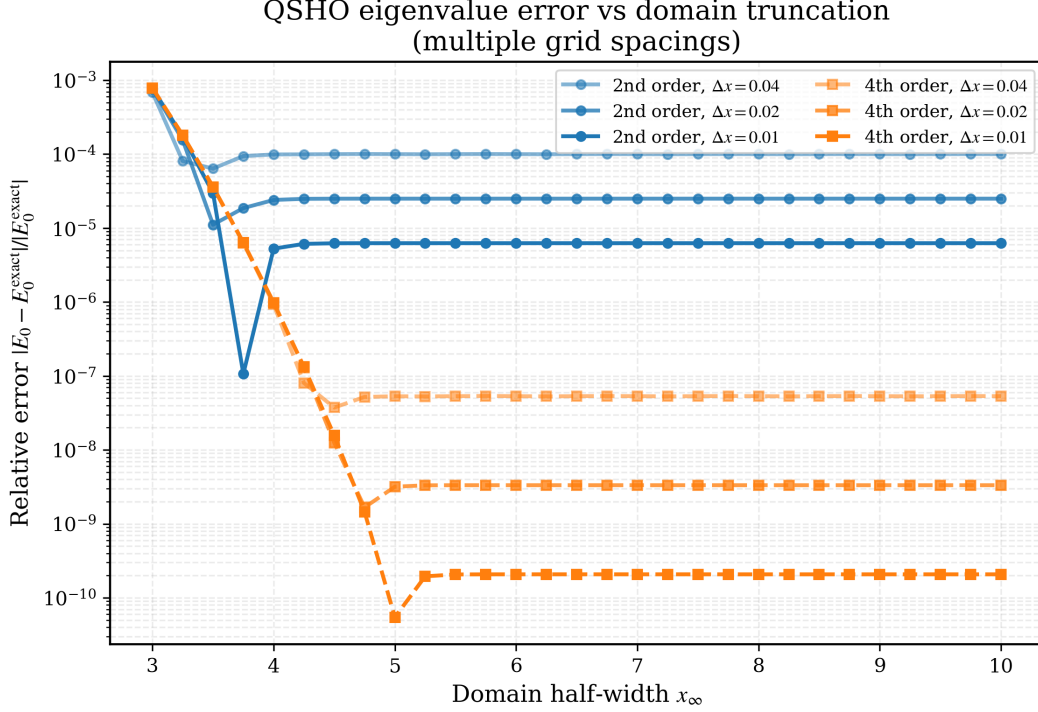


Figure 3.7: QSHO eigenvalue error versus domain half-width x_∞ , illustrating truncation sensitivity. Detailed plotting specifications are listed in Section A.5.

As expected, increasing x_∞ initially reduces truncation-driven bias, after which both curves enter a near-flat regime where discretization/solver effects dominate. In this dataset the high- x_∞ end shows a slight upward drift rather than a perfectly flat plateau, so the practical criterion is to choose the smallest x_∞ in the low-variation region.

For the present setup, the second-order curve transitions earlier to a higher error floor, while the fourth-order curve reaches a much lower minimum before the slight high- x_∞ drift appears. This separation is consistent with fourth-order reducing interior truncation error more effectively, thereby shifting the practical balance point between domain and discretization error.

In practical terms, the truncation study provides a calibration rule for subsequent production runs: first choose x_∞ at the start of the low-variation region (to suppress boundary-induced bias efficiently), then spend additional computational budget on grid refinement and solver robustness rather than further domain expansion.

Chapter 4

Conclusions and Future Work

4.1 Summary of Results

Chapters 1 and 2 established two central expectations: first, that higher-order finite differences should reduce truncation error more rapidly under grid refinement when boundary rows are treated consistently; second, that higher-energy states should be more sensitive to discretization because their oscillation scales are shorter. The numerical results in Chapter 3 support both expectations. Across ISW and QSHO benchmarks, fourth-order discretization delivered substantially lower errors at matched resolution, while mode-index analyses showed increasing error toward higher modes, consistent with the predicted node-density effects.

The runtime and truncation studies also clarified practical method selection. Runtime trends over the tested range favored the fourth-order configuration in the present solver setup, though the fitted slopes indicate that implementation and eigensolver behavior influence observed scaling beyond simple stencil-width arguments. For QSHO truncation, increasing x_∞ reduced bias up to a low-variation regime; beyond that region, domain expansion gave limited additional benefit compared with improving resolution or solver settings.

Taken together, these results indicate that fourth-order finite differences provide the strongest accuracy-per-degree-of-freedom performance in this framework, provided boundary closures are implemented carefully and solver behavior is validated in the relevant parameter regime.

4.2 Future Work

Several targeted extensions would strengthen and complete the analysis:

- Extend convergence and runtime sweeps to larger N to test high- N behavior more decisively, including possible crossover trends and memory-limit effects.
- Perform controlled solver-parameter sensitivity studies (tolerances, iteration limits, shift strategies) to isolate the source of high- N non-monotone error behavior.
- Add explicit log-log fit diagnostics (with confidence intervals) for all convergence curves in the final reported figures.
- Introduce quantitative wavefunction-error metrics (e.g., norm-based profile discrepancies) before reintroducing full wavefunction-comparison conclusions in the results chapter.
- Expand cross-benchmark spectral-error comparisons with matched settings to separate discretization effects from domain-truncation effects more cleanly.

Appendix A

Implementation Details

A.1 Implementation Details for Fourth-Order Boundary Corrections

A.1.1 Implementation Considerations

The boundary correction implementation ensures that:

- The interior stencil achieves fourth-order accuracy ($\mathcal{O}(\Delta^4)$ truncation error) using a five-point centered formula.
- The *boundary rows are not assumed* to be fourth-order; instead, the first two and last two interior rows are replaced by boundary closures derived from Taylor expansions under $\psi_0 = \psi_{N+1} = 0$.
- The first and last interior rows require a wider closure than five points: a six-point boundary closure is used at the boundary-adjacent rows to achieve global fourth-order accuracy without ghost nodes.
- When $B(x) \equiv 0$, the continuous operator is Hermitian; in the discrete fourth-order implementation, boundary-closure rows can introduce slight asymmetry, so robust workflows may use either Hermitian or general sparse eigensolver settings depending on the assembled matrix.

This boundary correction approach is crucial for maintaining the fourth-order accuracy throughout the entire domain, not just in the interior region. Without proper boundary corrections, the method would degrade to lower-order accuracy at the boundaries, significantly impacting the overall solution quality.

A.1.2 Implementation Algorithm

The boundary correction implementation follows these steps:

1. Assemble only the *interior* rows ($i = 3, \dots, N - 2$ in grid indexing) using the standard five-point stencil, producing a pentadiagonal sparse matrix on the interior block.
2. Replace the first two and last two interior rows with Taylor-derived closures that explicitly incorporate $\psi_0 = \psi_{N+1} = 0$:
 - boundary-adjacent rows (first/last interior point): six-point closure to maintain $\mathcal{O}(\Delta^4)$ accuracy without ghost nodes,
 - near-boundary rows (second/second-to-last interior point): shifted five-point centered closure, with boundary terms eliminated by $\psi_0 = \psi_{N+1} = 0$.
3. Insert these boundary-row coefficients directly into the sparse matrix construction (COO/CSR), so the final operator includes the correct boundary treatment by construction.

This approach ensures that the boundary corrections are applied only where needed, maintaining the pentadiagonal structure while achieving fourth-order accuracy throughout the domain.

A.1.3 Matrix Verification Methodology

To ensure correctness of the implementation, we perform matrix verification by manually calculating the expected matrix for a small test case ($N=7$) and comparing it with the computed matrix. This verification process involves:

1. Choosing specific functions for $A(x)$, $B(x)$, and $C(x)$
2. Manually calculating the expected matrix entries using the finite difference formulas
3. Comparing the computed matrix with the expected matrix element by element

This verification is crucial for catching implementation errors that might not be apparent in convergence tests.

A.2 Unified Implementation Architecture

The refactored implementation uses a completely unified approach that eliminates code duplication and provides a single, extensible framework for finite difference methods of any order. Modern computational tools like NumPy and SciPy provide efficient implementations of the underlying linear algebra operations [10, 15].

A.2.1 Coefficient Management System

The implementation centralizes finite difference coefficients through a unified coefficient function:

- `_get_coefficients(order)`: Returns coefficient arrays for any order (2, 4, or easily extensible to 6)
- `coeffs_d2`, `coeffs_d1`, `coeffs_d0`: Arrays for second, first, and zeroth derivative terms
- Automatic coefficient scaling and indexing for different stencil sizes

A.2.2 Unified Diagonal Construction

All matrix diagonals are constructed using a single, order-agnostic function:

- `_diagonal_unified(A, B, C, grid, k, order=2)`: Constructs diagonal entries for any offset k and any order
- Mathematical indexing: $k \in [-order/2, +order/2]$ with automatic boundary handling
- Unified coefficient lookup: $idx = k + order//2$ for correct array indexing
- Handles both 2nd-order (tridiagonal) and 4th-order (pentadiagonal) stencils

A.2.3 Unified Matrix Assembly

The matrix assembly process is completely unified through a single function:

- `sparse_Matrix_Maker(A, B, C, grid, order=2)`: Single function for all orders
- Loop structure: `for k in range(-order//2, order//2 + 1)` to build all diagonals
- Automatic boundary correction integration based on order parameter
- Backward compatibility: `sparse_Matrix_Maker4(A, B, C, grid)` calls unified function with `order=4`

A.2.4 Unified Boundary Corrections

Boundary corrections are handled through a single, order-aware function:

- `_boundary_corrections(A, B, C, grid, order=4)`: Order-aware boundary correction function
- Order-specific logic: Returns empty arrays for `order=2`, applies closures for `order=4`
- For `order=4`, the first/last interior rows use a ghost-free *six-point* closure (five nonzeros on each of those rows in the interior-only matrix), and the second/second-to-last rows use a shifted *five-point* closure (four nonzeros because $\psi_0 = \psi_{N+1} = 0$)
- The coefficient patterns match Equations (1.35) to (1.38) and are injected directly during sparse COO assembly

A.2.5 Unified Hamiltonian Construction

Quantum mechanical hamiltonians are constructed through a single, order-parameterized function:

- `hamiltonian(V, x_left, x_right, N, order=2)`: Unified hamiltonian function for all orders
- Automatic coefficient setup: $A(x) = -0.5$, $B(x) = 0.0$, $C(x) = V(x)$
- Backward compatibility: `hamiltonian_4th(V, x_left, x_right, N)` calls unified function with `order=4`
- Single codebase: All hamiltonian construction uses the same underlying matrix assembly

A.2.6 Code Architecture Benefits

The unified implementation provides several key advantages:

1. **Extensibility**: Adding 6th-order methods requires only updating coefficient arrays
2. **Maintainability**: Single codebase eliminates duplication and reduces bugs
3. **Consistency**: All methods use identical matrix assembly logic
4. **Performance**: Unified approach optimizes memory usage and computation
5. **Backward Compatibility**: Existing code continues to work without modification

A.2.7 Implementation Algorithm

The complete matrix construction follows this unified algorithm, leveraging efficient sparse matrix operations [10]:

1. Initialize empty arrays for `rows`, `cols`, `data`
2. Loop over all diagonal offsets: `for k in range(-order//2, order//2 + 1)`
3. For each k , call `_diagonal_unified(A, B, C, grid, k, order)` to get diagonal entries
4. Concatenate diagonal results to build the base matrix
5. Add boundary corrections using `_boundary_corrections(A, B, C, grid, order)`
6. Construct final sparse matrix in COO format
7. Convert to CSR format for efficient linear algebra operations

This unified architecture ensures that all finite difference methods share the same robust, well-tested implementation while maintaining the flexibility to handle different orders and boundary conditions.

A.3 ISW Implementation

The Infinite Square Well implementation follows the same unified architecture as the QSHO, utilizing the `hamiltonian()` function with the ISW potential. The key difference is the potential function definition:

```
def V_ISW(x):
    """Infinite Square Well potential (zero inside, infinite outside)"""
    return np.zeros_like(x)
```

The dimensionless domain $[0, 1]$ with homogeneous Dirichlet boundary conditions provides an ideal test case for validating the finite difference method's accuracy and convergence properties.

A.4 Python Module Summaries

For reproducibility, this section records the core routines referenced throughout the thesis. Each subsection lists the relevant functions in context and provides short code excerpts to anchor the discussion in Chapters 1 and 2.

A.4.1 `finite_difference_quantum.py`

The grid helper `spatial()` generates the interior mesh and is paired with `proj()`, which enforces complex-valued sampling so that non-Hermitian test problems can be accommodated without refactoring:

```
def spatial(x_left, x_right, N):
    return np.linspace(x_left, x_right, N + 2)[1:-1]

def proj(f, grid):
    return np.array([f(point) for point in grid], dtype=complex)
```

Matrix construction is unified through a single dispatcher that accepts an `order` flag. The fourth-order path automatically appends boundary corrections computed in `_boundary_corrections()`:

```
def sparse_Matrix_Maker(A, B, C, grid, order=2):
    rows = cols = data = np.array([])
    for k in range(-order//2, order//2 + 1):
        r, c, d = _diagonal_unified(A, B, C, grid, k, order=order)
        rows = np.concatenate([rows, r])
        cols = np.concatenate([cols, c])
        data = np.concatenate([data, d])
    rBC, cBC, dBC = _boundary_corrections(A, B, C, grid, order=order)
    rows = np.concatenate([rows, rBC])
    cols = np.concatenate([cols, cBC])
    data = np.concatenate([data, dBC])
    return sparse.coo_matrix((data, (rows, cols)), shape=(len(grid), len(grid)))
```

Quantum Hamiltonians invoke the same builder, ensuring both orders share an identical code path apart from coefficient stencils:

```
def hamiltonian(V, x_left, x_right, N, order=2):
    grid = spatial(x_left, x_right, int(N))
    def A(x): return -0.5
    def B(x): return 0.0
    def C(x): return V(x)
    return sparse_Matrix_Maker(A, B, C, grid, order=order).tocsr()
```

A.4.2 eigenvalue_analysis.py

This module couples the Hamiltonian builders with SciPy's Lanczos eigensolver. Analytical reference spectra are provided for both benchmarks, and numerical counterparts are gathered in single calls to `eigsh`:

```
def numerical_ISW_eigenvalues(n_eigenvalues, n_grid_points, order=4, L=1.0):
    def V_ISW(x): return np.zeros_like(x)
    H = fdq.hamiltonian(V_ISW, 0.0, L, n_grid_points, order=order)
    eigenvalues, _ = eigsh(H, k=n_eigenvalues, which='SM', tol=1e-10, maxiter=10000)
    return np.sort(eigenvalues)
```

Post-processing utilities return absolute and relative error matrices together with metadata (grid spacing, domain size, order) that drive the convergence and runtime plots planned for Chapter 3:

```
def analyze_ISW_accuracy(n_eigenvalues, n_grid_points, order=4, L=1.0):
    analytical = analytical_ISW_eigenvalues(n_eigenvalues, L=L)
    numerical = numerical_ISW_eigenvalues(n_eigenvalues, n_grid_points, order=order, L=L)
    abs_errors, rel_errors = compute_eigenvalue_discrepancies(numerical, analytical)
    dx = L / (n_grid_points + 1)
    return {"analytical": analytical, "numerical": numerical,
            "absolute_errors": abs_errors, "relative_errors": rel_errors,
            "grid_spacing": dx, "n_grid_points": n_grid_points, "order": order}
```

A.4.3 n_break_test.py

The stress-test script incrementally doubles the grid size until a memory error is encountered, logging the final successful value for both second- and fourth-order operators. These diagnostics inform the runtime discussion in Chapter 3:

```
def test_order(order):
    n = 10
    while True:
        try:
            grid = spatial(x_left, x_right, n)
            if order == 2:
                sparseMatrixMaker(A, B, C, grid)
            else:
                sparse_Matrix_Maker4(A, B, C, grid)
            with open(f"n_break_order{order}.txt", "a") as f:
                f.write(f"{n}\n")
            n *= 2
        except MemoryError:
            break
```

Together, these listings provide the executable backbone for the analytical narrative, ensuring that every claim in the main text corresponds to verifiable source code.

A.5 Figure Plotting Specifications

This section consolidates plotting/export specifications for all figures whose main-text captions point here.

Table A.1: Chapter 2 figure plotting specifications (compact reference).

Figure	File	Key specifications
2.2	<code>isw_analytical_spectrum.png</code>	ISW on $\tilde{x} \in [0, 1]$; levels $\tilde{E}_n = n^2\pi^2$ for $n = 1, \dots, 7$; black potential + red horizontal level lines (lw 1.8); serif/STIX; dashed major grid ($\alpha = 0.25$); $\sim 7.2 \times 4.2$ in; 300 dpi.
2.3	<code>isw_eigenfunctions_analytic.png</code>	1200-point centered grid $\tilde{x} \in [-0.5, 0.5]$; $n = 1..4$ modes from $\sqrt{2}\sin[n\pi(\tilde{x} + \frac{1}{2})]$; styles: solid/dashed/dash-dot/dotted; lw 1.9; serif/STIX; dashed major grid ($\alpha = 0.25$); $\sim 6.8 \times 4.2$ in; 300 dpi.
2.4	<code>images/qsho_potential_turning_points.png</code>	QSHO $\tilde{V} = \tilde{x}^2$ on $\tilde{x} \in [-4, 4]$; highlighted $n = 0, 3$ levels; dashed level guides + dotted turning-point lines $\tilde{x} = \pm\sqrt{\tilde{E}_n}$; curve lw 2.0, guide lw 2.0/1.4; serif/STIX; $\sim 7.0 \times 4.0$ in; 300 dpi.
2.5	<code>qsho_analytical_spectrum.png</code>	QSHO potential on $\tilde{x} \in [-4.2, 6.3]$; levels $\tilde{E}_n = 2n + 1$ for $n = 0, \dots, 7$ between turning points; black potential + red level segments (lw 1.8); serif/STIX; dashed major grid ($\alpha = 0.25$); $\sim 7.2 \times 4.2$ in; 300 dpi.
2.6	<code>qsho_eigenfunctions_analytic.png</code>	Dense uniform grid $\tilde{x} \in [-4, 4]$; $n = 0..3$ Hermite-Gaussian modes; styles solid/dashed/dash-dot/dotted; lw ≈ 1.9 ; legend upper right; serif/STIX; dashed major grid ($\alpha = 0.25$); 300 dpi.

Table A.2: Chapter 3 figure plotting specifications (compact reference).

Figure	File	Key specifications
3.1	<code>isw_runtime_only.png</code>	Single-panel ISW runtime plot with log-log axes, 2nd/4th-order measured curves, and linear-fit slope guides (m_2 , m_4); exported 300 dpi.
3.2	<code>qsho_runtime_measured_only.png</code>	QSHO fourth-order measured-to-limit runtime sweep on log-log axes, extended to practical N -break; linear fit of $\log T$ vs $\log N$ yields slope $m \approx 1.73$; exported 300 dpi.
3.3	<code>isw_convergence.png</code>	ISW ground-state relative error vs N on log-log axes for orders 2 and 4; geometric N sampling via configured run; optional clean-slope guide overlays; serif/STIX; dashed major grid; exported 300 dpi.
3.4	<code>qsho_convergence.png</code>	QSHO ground-state relative error vs N on log-log axes for orders 2 and 4; geometric N sampling via configured run; optional clean-slope guide overlays; serif/STIX; dashed major grid; exported 300 dpi.
3.5	<code>isw_numerical_vs_analytical_eigenvalues.png</code>	Two-panel ISW spectrum check: left panel analytical vs numerical eigenvalues, right panel mode-index relative error (log scale); consistent order coloring/markers; exported 300 dpi.
3.6	<code>qsho_numerical_vs_analytical_eigenvalues.png</code>	Two-panel QSHO spectrum check: left panel analytical vs numerical eigenvalues, right panel mode-index relative error (log scale); matched layout/styling with ISW benchmark figure; generated at fixed grid size ($N = 6000$ in the reported run); exported 300 dpi.
3.7	<code>qsho_error_vs_domain.png</code>	QSHO ground-state relative eigenvalue error vs domain half-width x_∞ with multiple fixed grid spacings ($\Delta x = 0.04, 0.02, 0.01$); both 2nd-order (blue circles, solid) and 4th-order (orange squares, dashed) are shown on log- y axes over $x_\infty \in [3, 10]$; dense uniform x_∞ sweep; serif/STIX styling with major dashed grid and compact two-column legend; exported at 300 dpi.

Bibliography

- ¹R. J. LeVeque, *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems* (SIAM, Philadelphia, PA, 2007).
- ²G. Strang, *Computational science and engineering* (Wellesley-Cambridge Press, Wellesley, MA, 2007).
- ³D. J. Griffiths and D. F. Schroeter, *Introduction to quantum mechanics*, 3rd (Cambridge University Press, Cambridge, UK, 2018).
- ⁴L. N. Trefethen and D. Bau, *Numerical linear algebra* (SIAM, Philadelphia, PA, 1997).
- ⁵Wolfram Research, *Finite difference*, <https://mathworld.wolfram.com/FiniteDifference.html>, Accessed: 2024, 2024.
- ⁶Physics 310 Instructor, *Lecture notes: finite difference methods*, Union College Physics 310, Lecture 9, 2024.
- ⁷B. Fornberg, “Generation of finite difference formulas on arbitrarily spaced grids”, *Mathematics of Computation* **51**, 699–706 (1988).
- ⁸Wikipedia contributors, *Finite difference coefficient*, https://en.wikipedia.org/wiki/Finite_difference_coefficient, Accessed: 2024, 2024.
- ⁹MIT OpenCourseWare, *Numerical methods for partial differential equations*, <https://ocw.mit.edu/courses/>, Massachusetts Institute of Technology, 2024.
- ¹⁰P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, *SciPy 1.11: fundamental algorithms for scientific computing in python*, 2023.
- ¹¹R. Shankar, *Principles of quantum mechanics*, 2nd (Springer, New York, NY, 2012).
- ¹²Physics Stack Exchange contributors, *Imaginary constant in simplification of the infinite square well problem*, Accessed: 16 Feb 2026, (2020) <https://physics.stackexchange.com/questions/544066/imaginary-constant-in-simplification-of-the-infinite-square-well-problem>.
- ¹³*Analysis of path integrals at low temperature: box formula, occupation time and ergodic approximation — scientific figure on ResearchGate*, https://www.researchgate.net/figure/Probability-density-of-the-ground-state-for-the-harmonic-oscillator-The-solid-line_fig5_2199537, Accessed: 15 Feb 2026.
- ¹⁴N. J. Higham, *Accuracy and stability of numerical algorithms*, 2nd (SIAM, Philadelphia, PA, 2002).
- ¹⁵C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, *NumPy 1.26: fundamental package for array computing in python*, 2023.